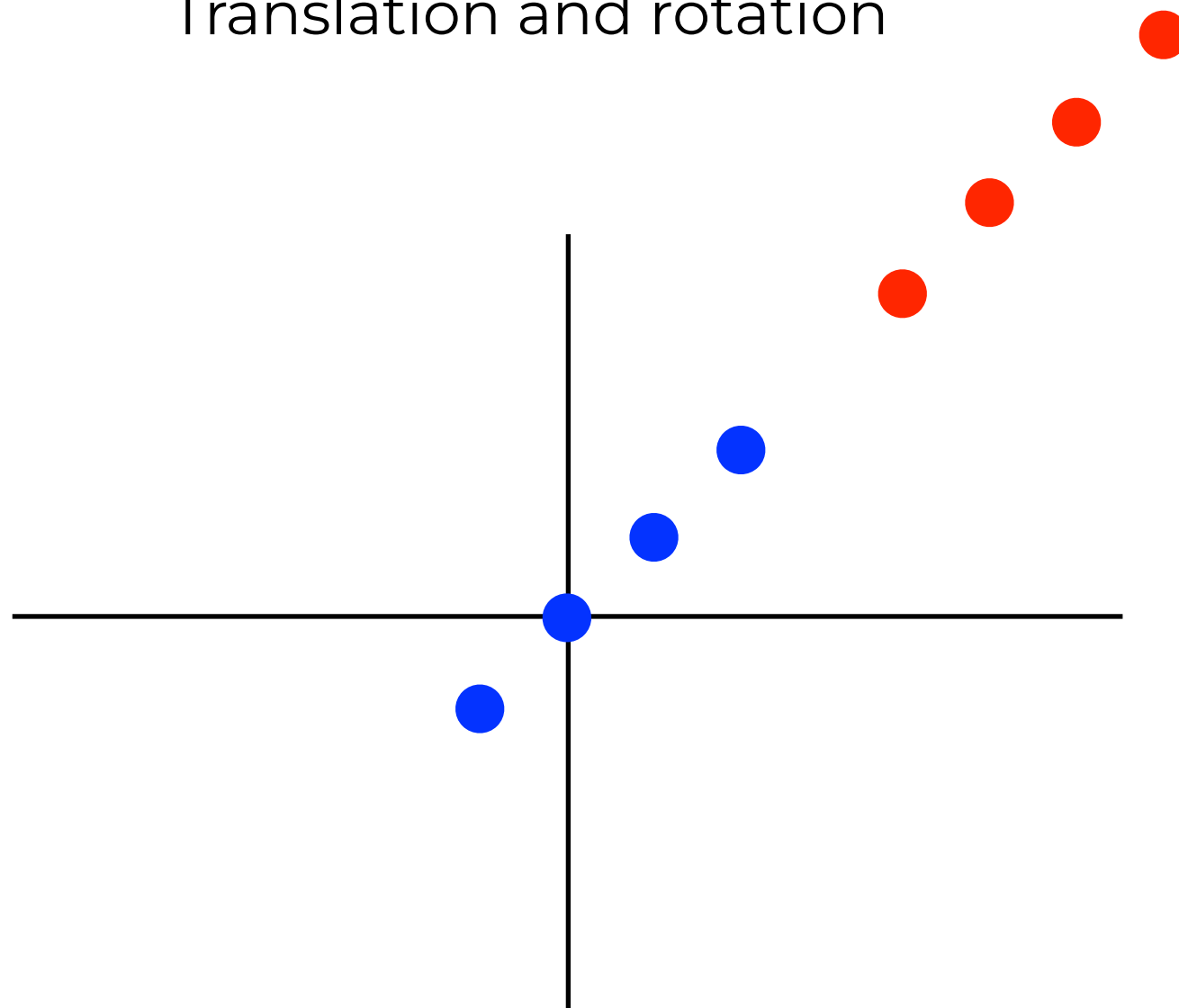


Principal Component Analysis (PCA)

Itthi Chatnuntawech

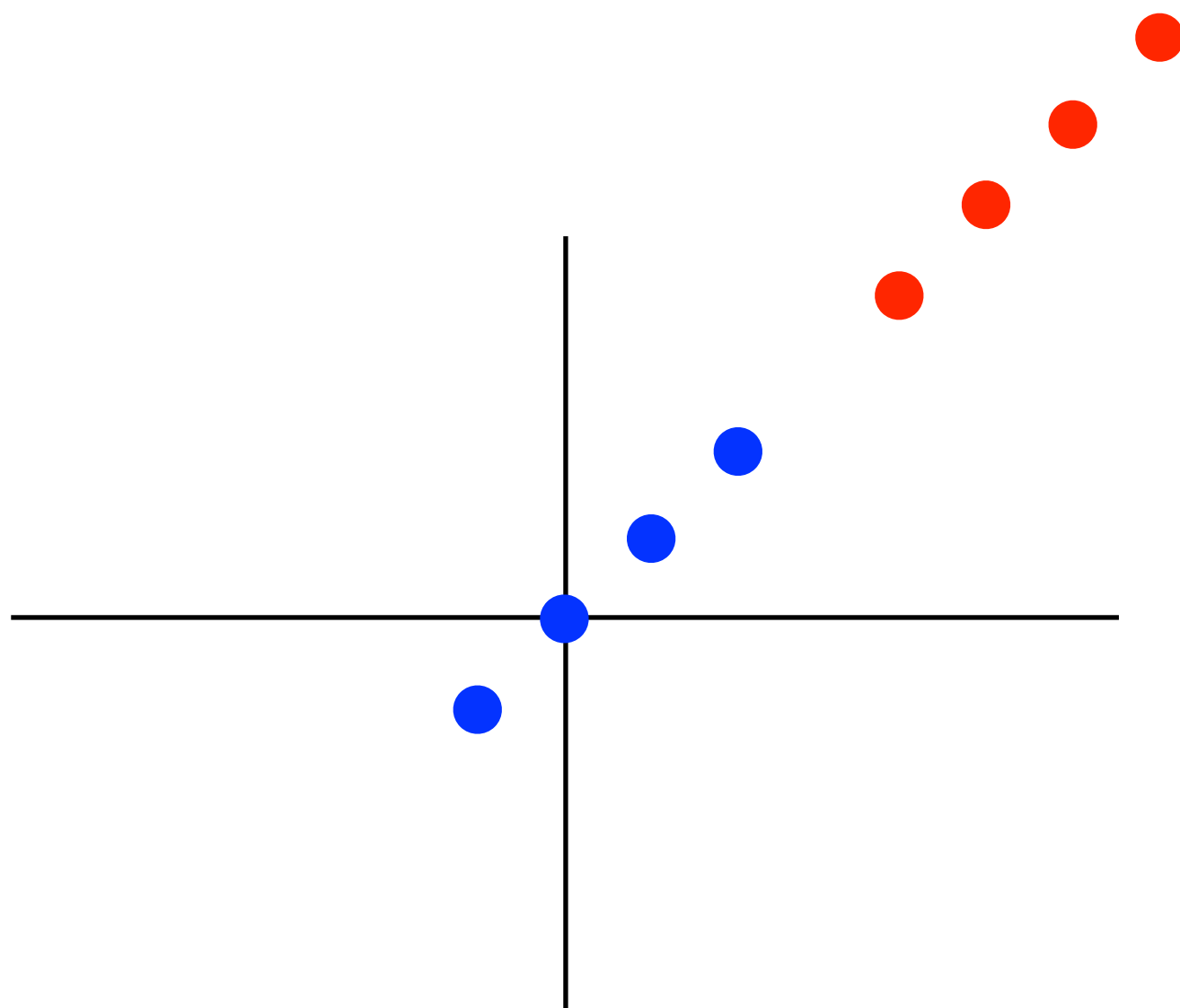
Simple Transformations

Translation and rotation

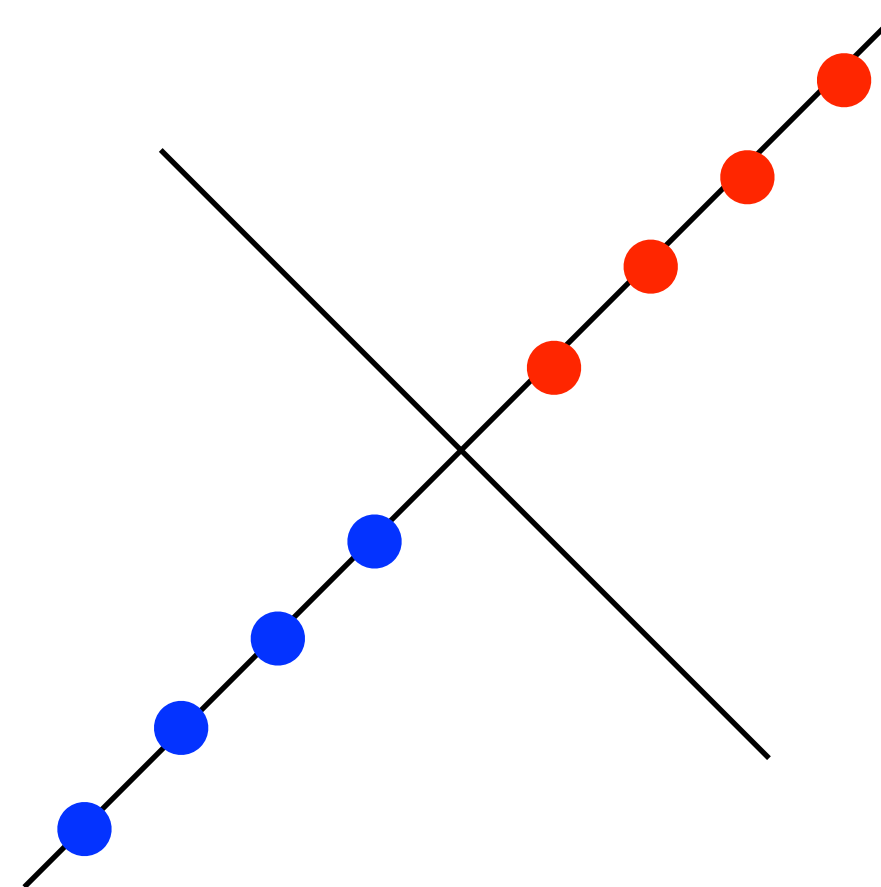


Simple Transformations

Original space

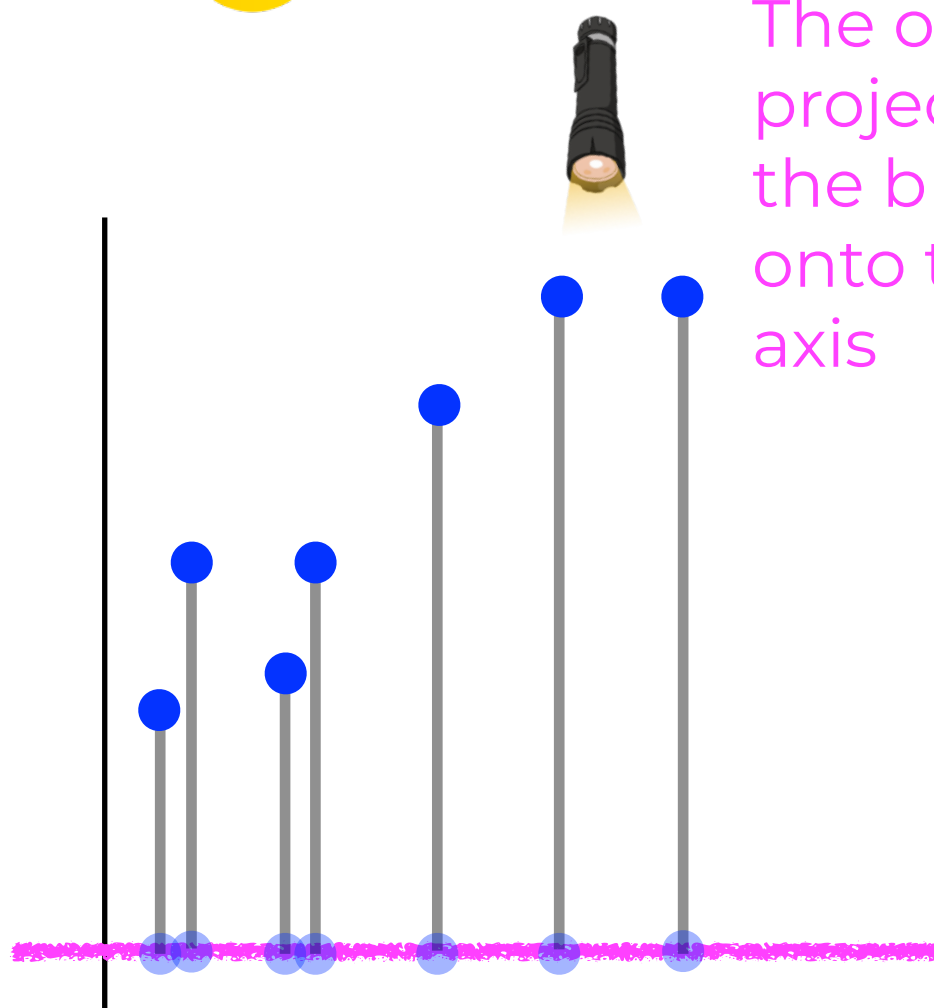


Transformed space

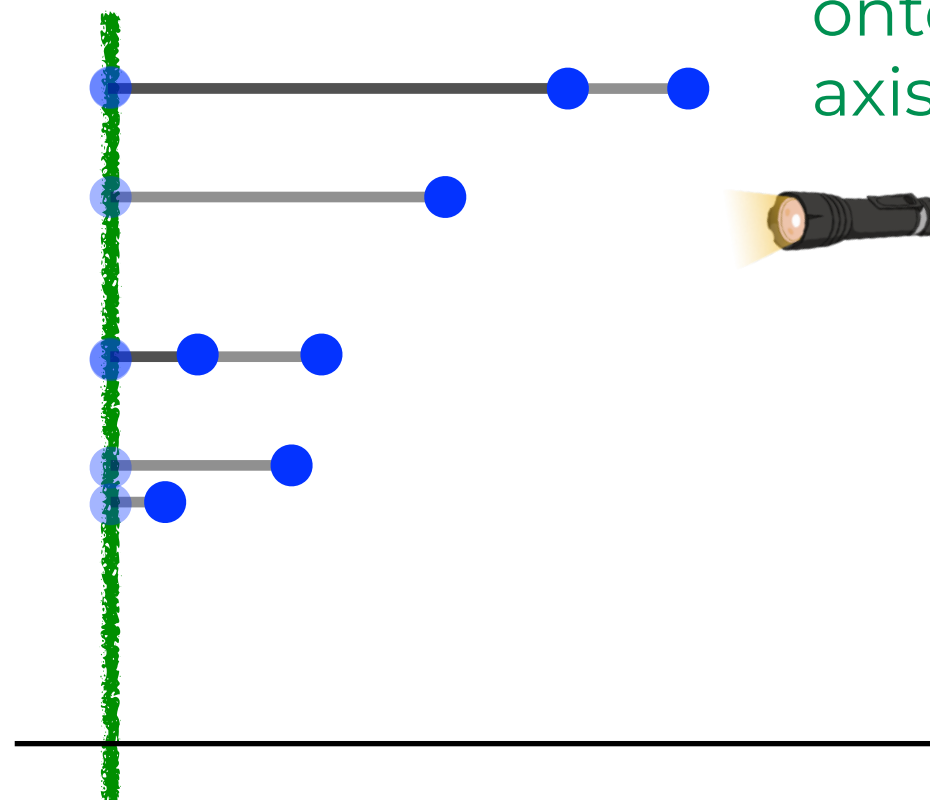


Projection

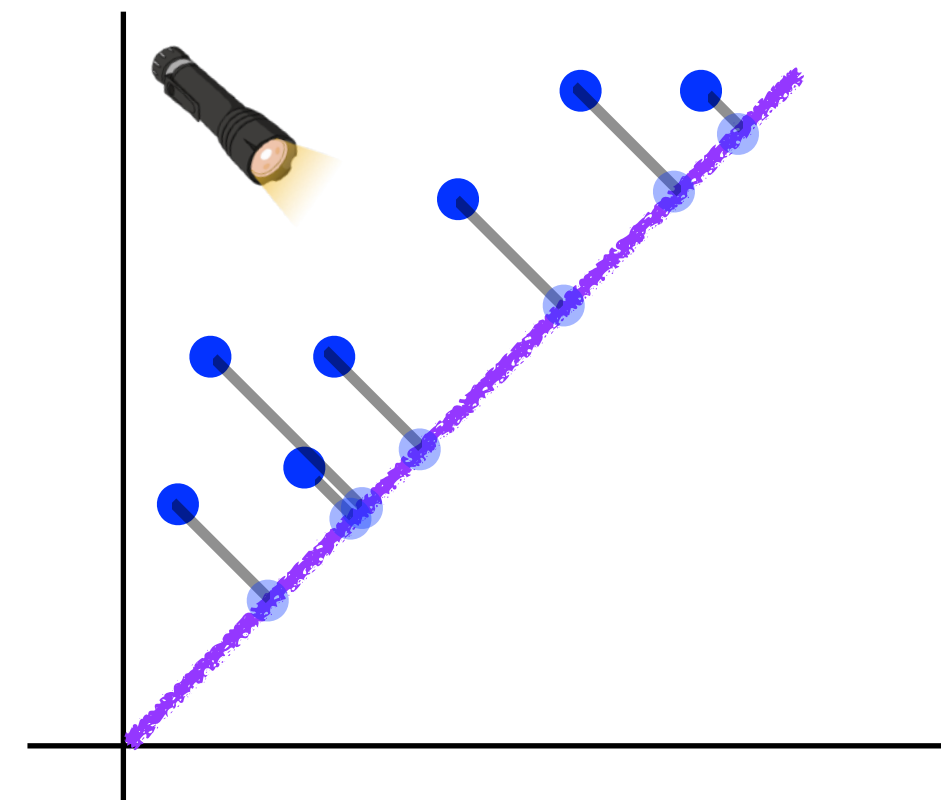
The orthogonal projection of the blue points onto the pink axis



The orthogonal projection of the blue points onto the green axis

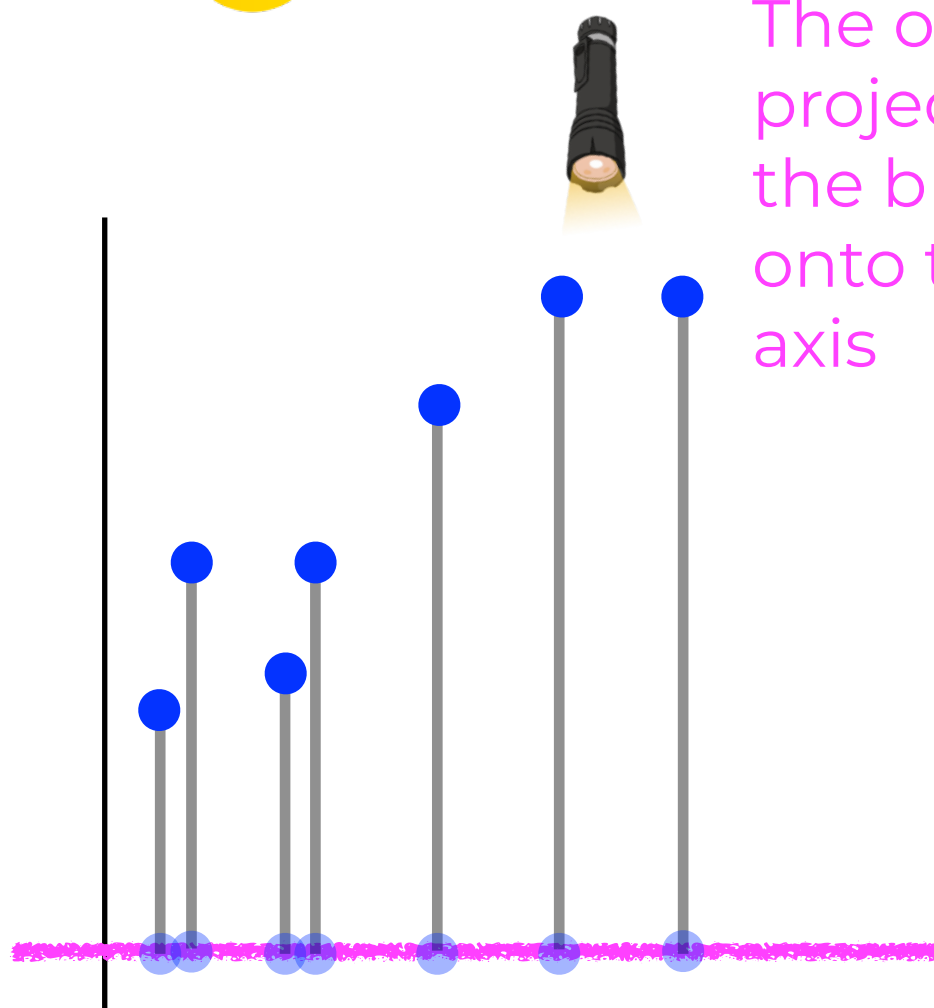


The orthogonal projection of the blue points onto the purple axis



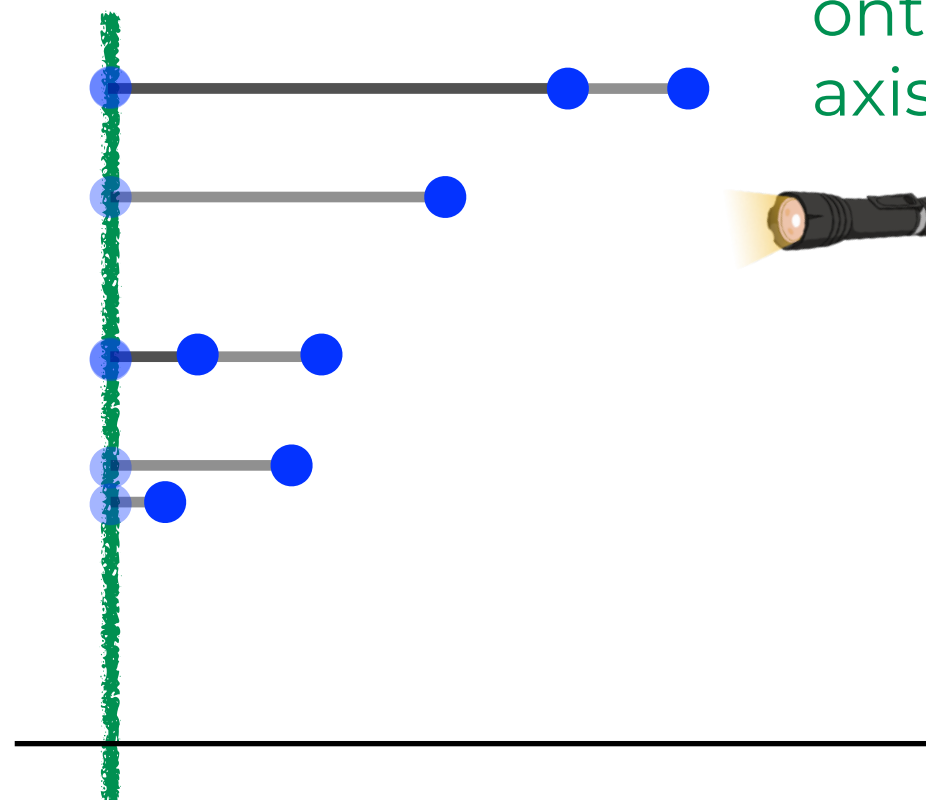
Projection

The orthogonal projection of the blue points onto the pink axis

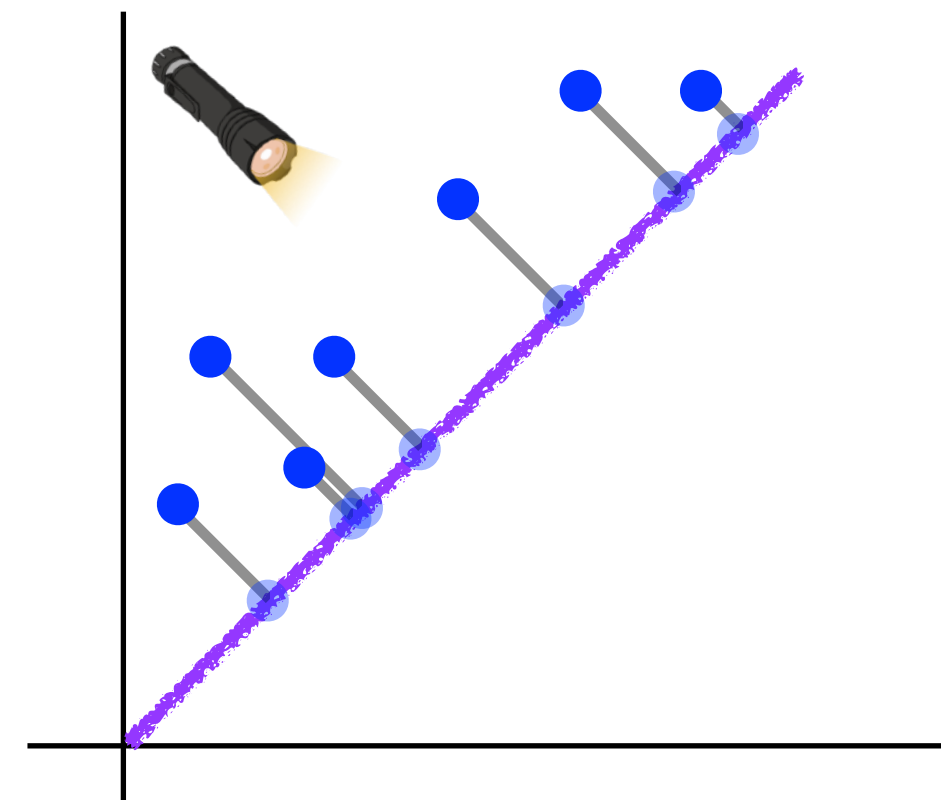


How spread out the projected points are

The orthogonal projection of the blue points onto the green axis



The orthogonal projection of the blue points onto the purple axis



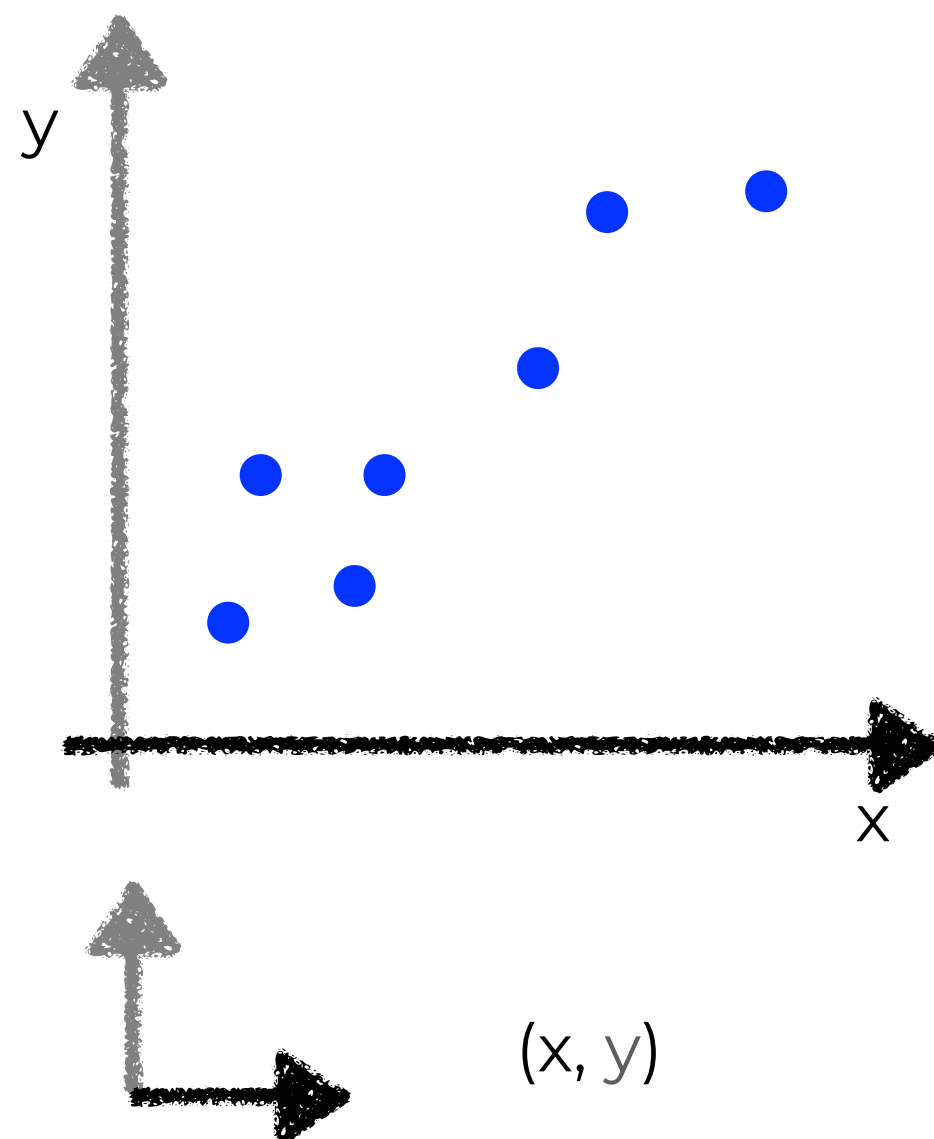
higher



lower

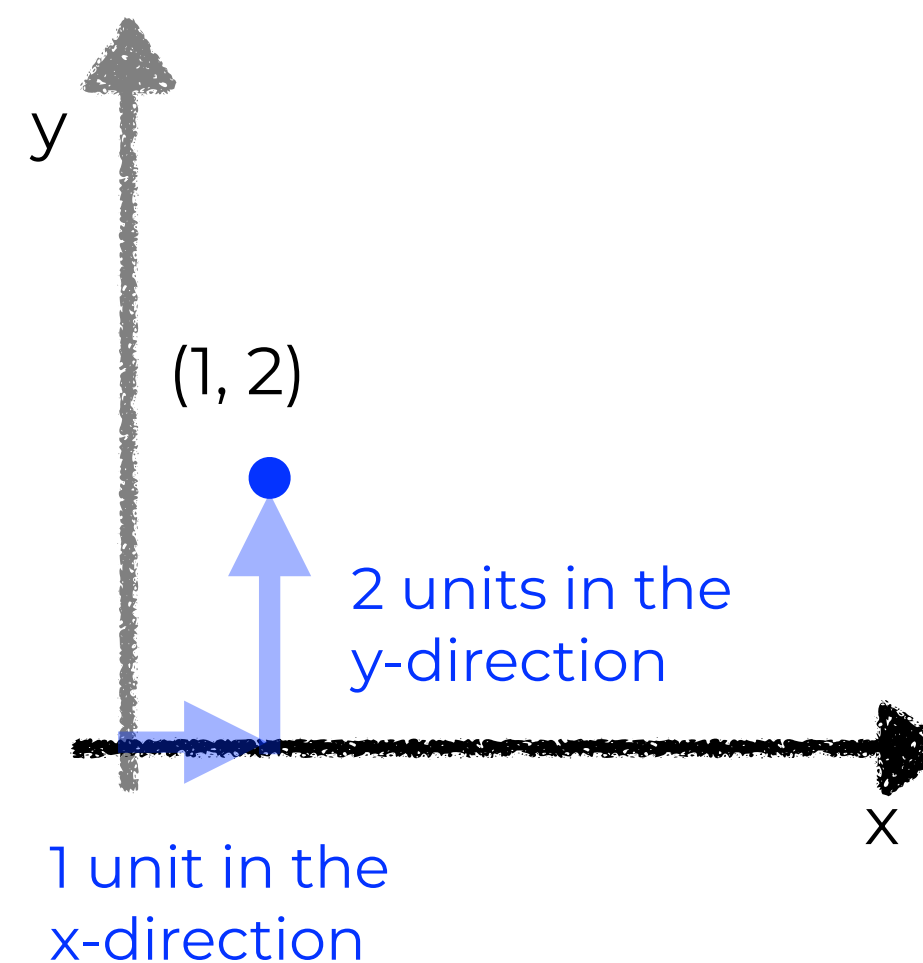
Principal Component Analysis (PCA)

Original space



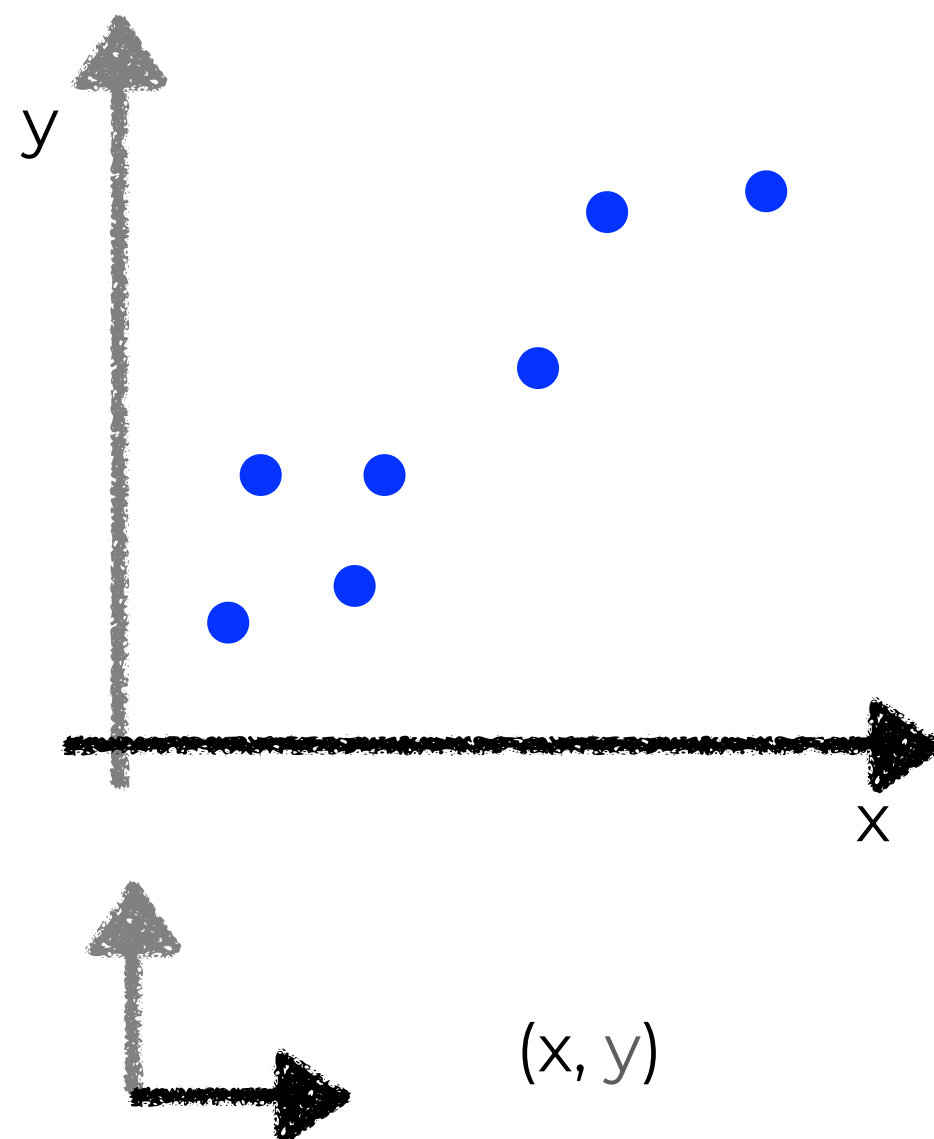
How much you
move in the x-
direction

How much you
move in the y-
direction



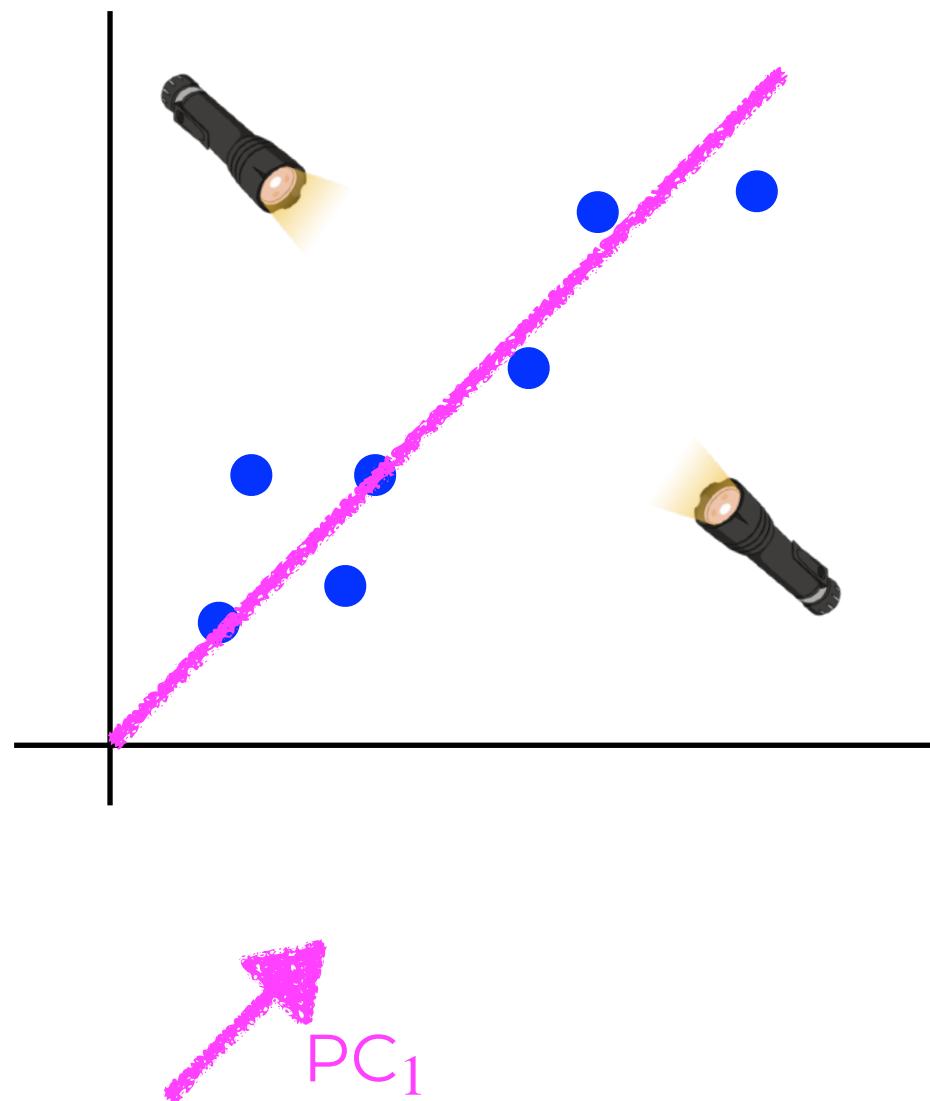
Principal Component Analysis (PCA)

Original space



How much you
move in the x-
direction

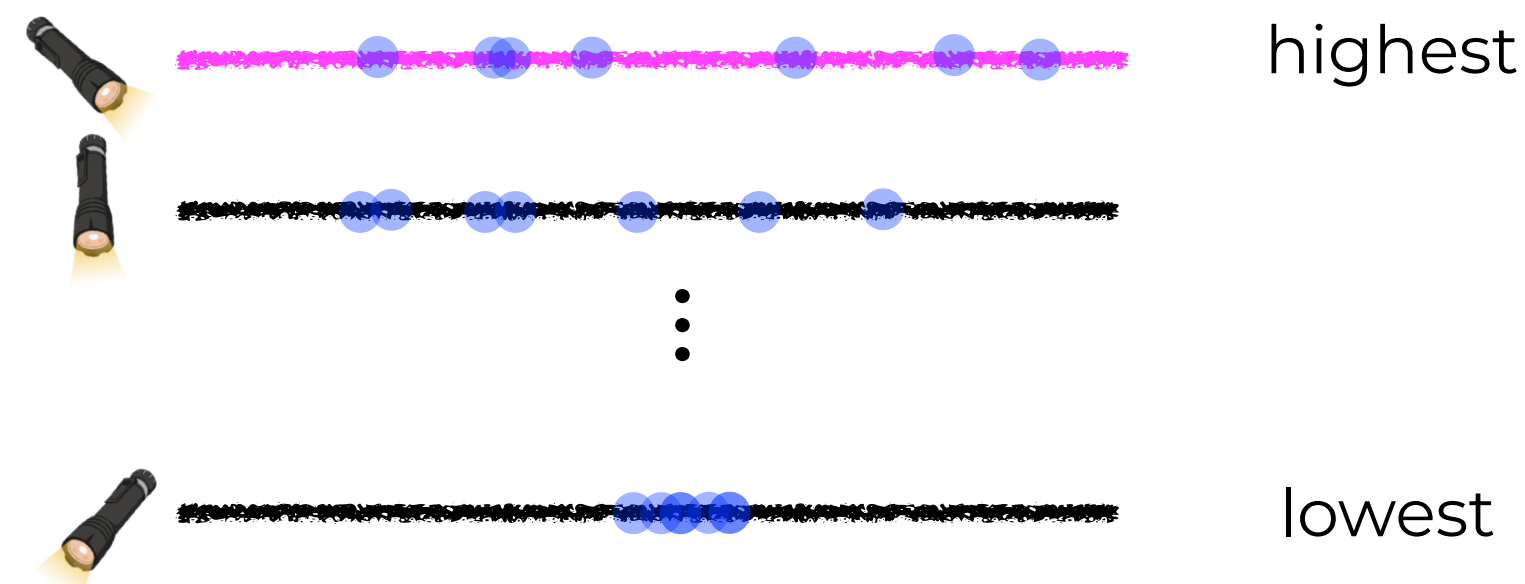
How much you
move in the y-
direction



For d-dimensional data, PCA can find up to d directions (principal components or PCs)

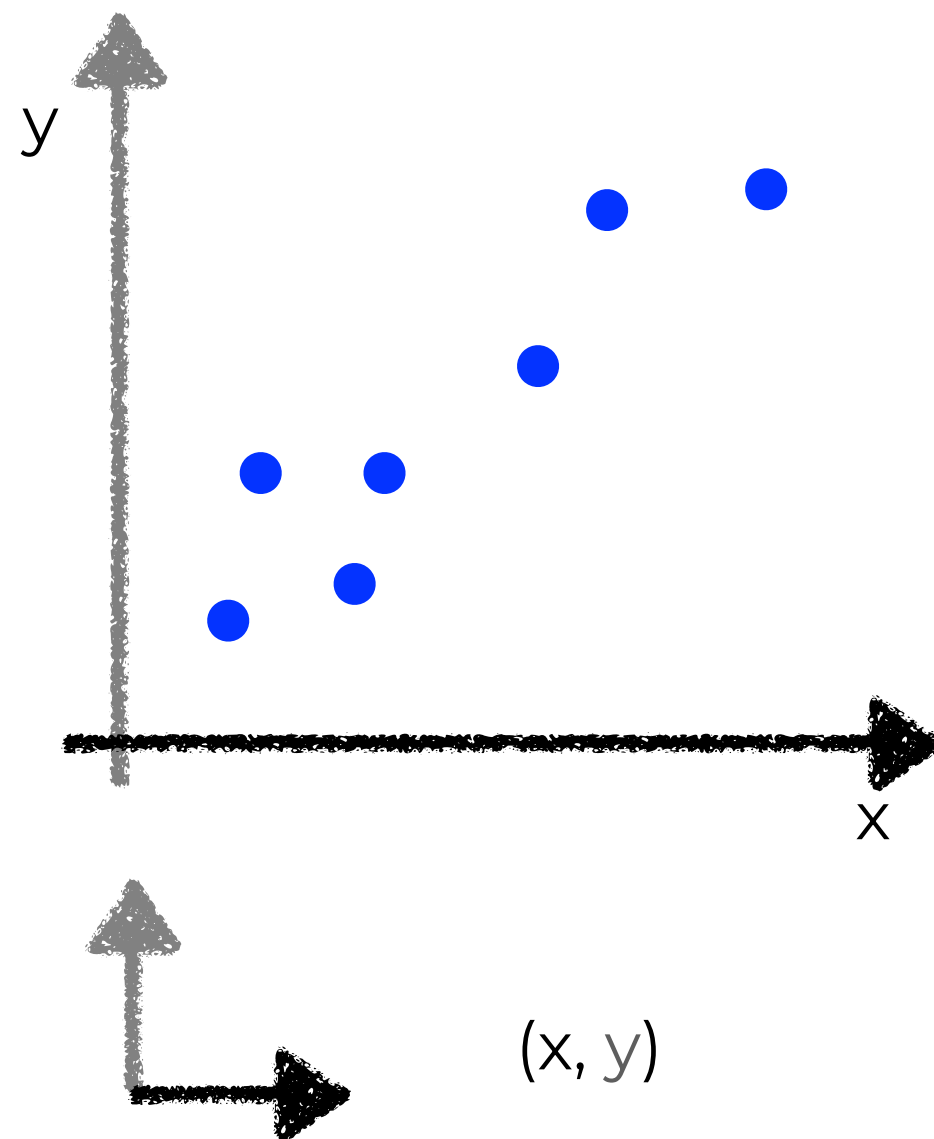
1. Find the direction that **maximizes the variance** of the projected points. This direction can be described by a vector called **the first principal component (PC₁)**

How spread out the
projected points are



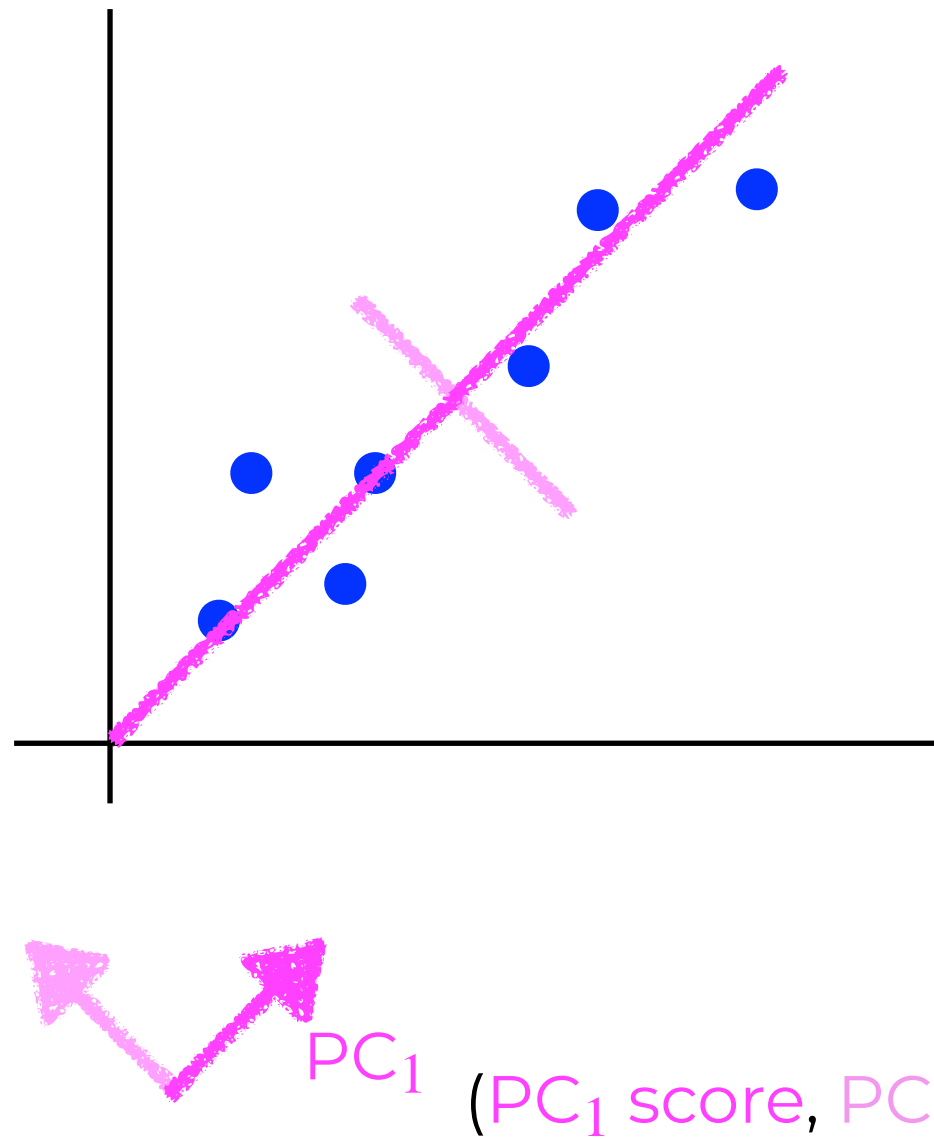
Principal Component Analysis (PCA)

Original space



How much you
move in the x-
direction

How much you
move in the y-
direction



How much you
move in the PC₁
direction

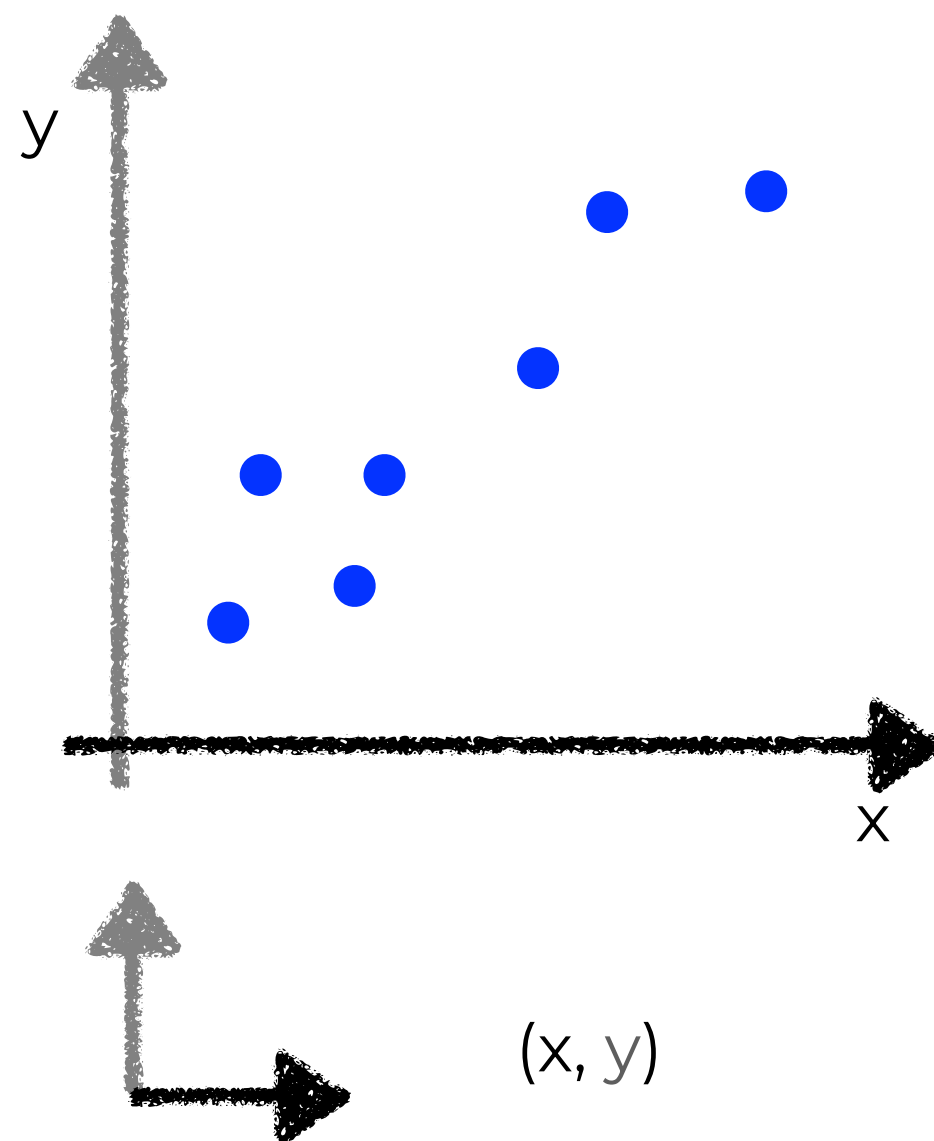
How much you
move in the PC₂
direction

For d-dimensional data, PCA can find up to d directions (principal components or PCs)

1. Find the direction that **maximizes the variance** of the projected points. This direction can be described by a vector called **the first principal component (PC₁)**
2. Find the next PC that
 - is **orthogonal** to the PCs already considered
 - **maximizes the variance** along the new direction
3. Repeat step 2 until you have the desired number of PCs

Principal Component Analysis (PCA)

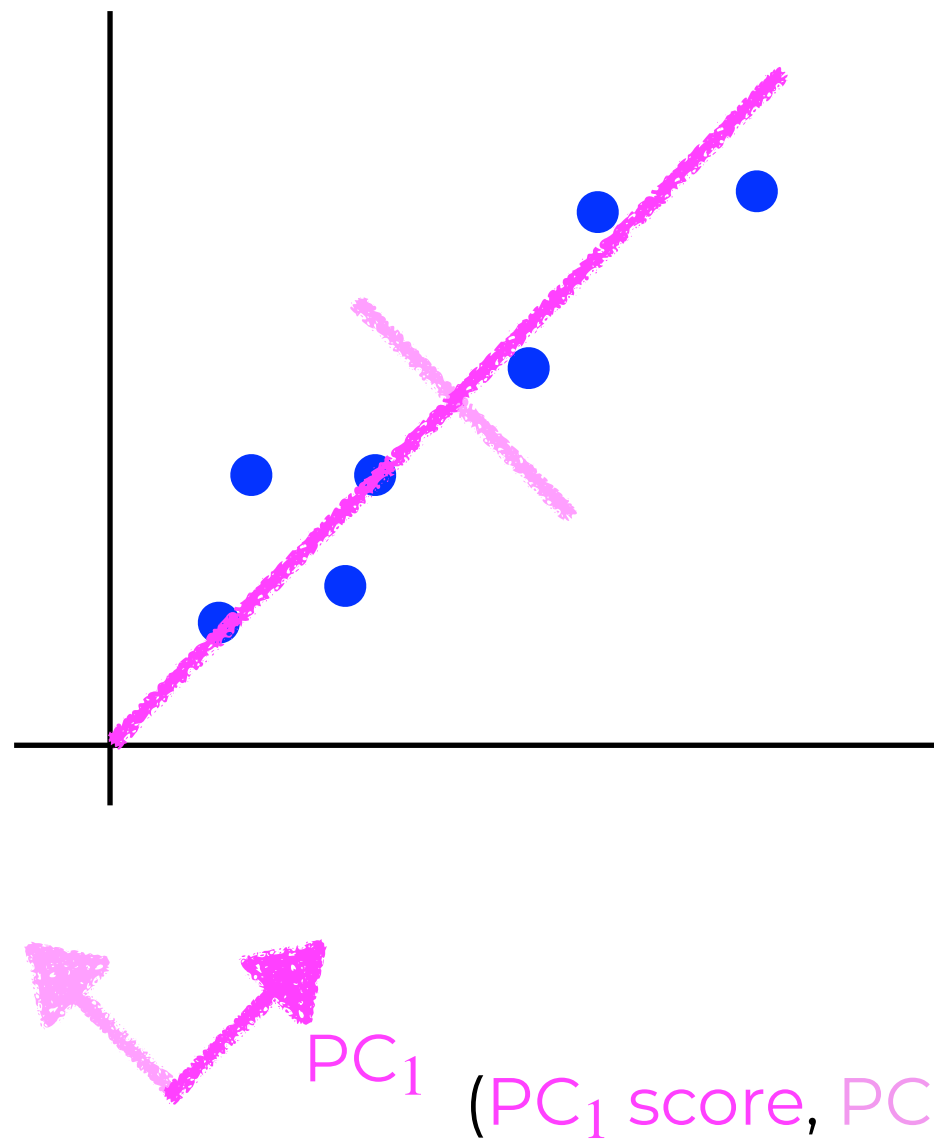
Original space



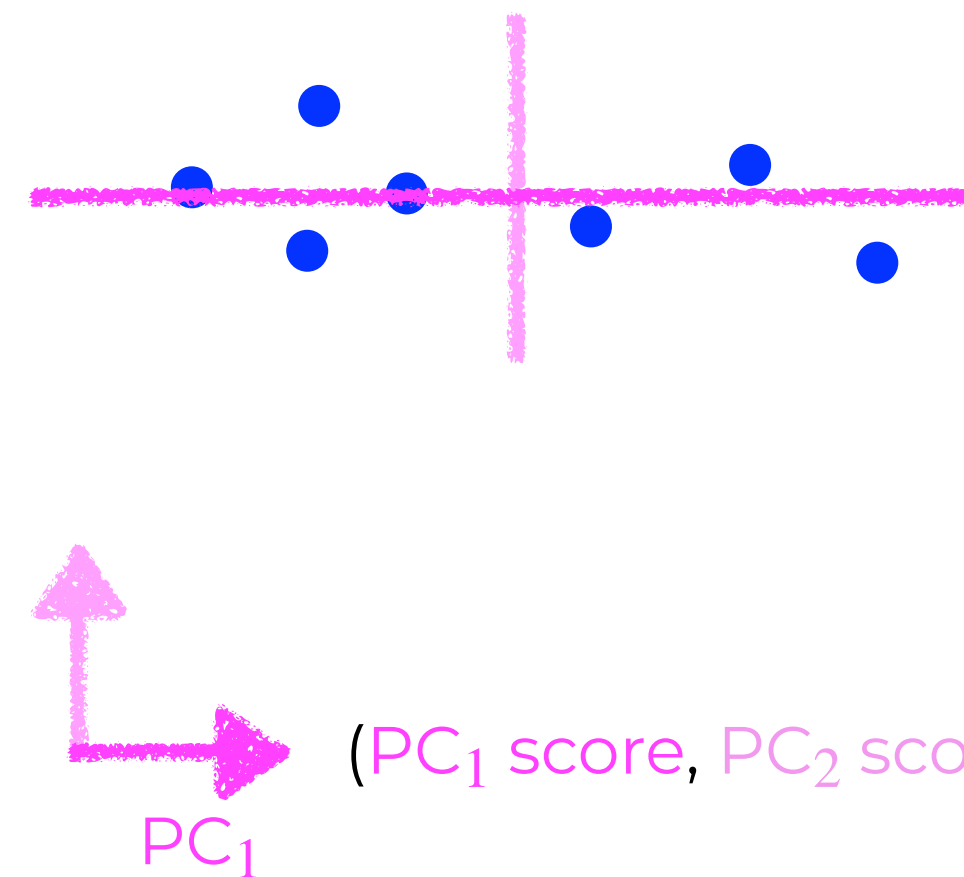
How much you
move in the x-
direction

How much you
move in the y-
direction

PCA



PC_1 $(PC_1 \text{ score}, PC_2 \text{ score})$



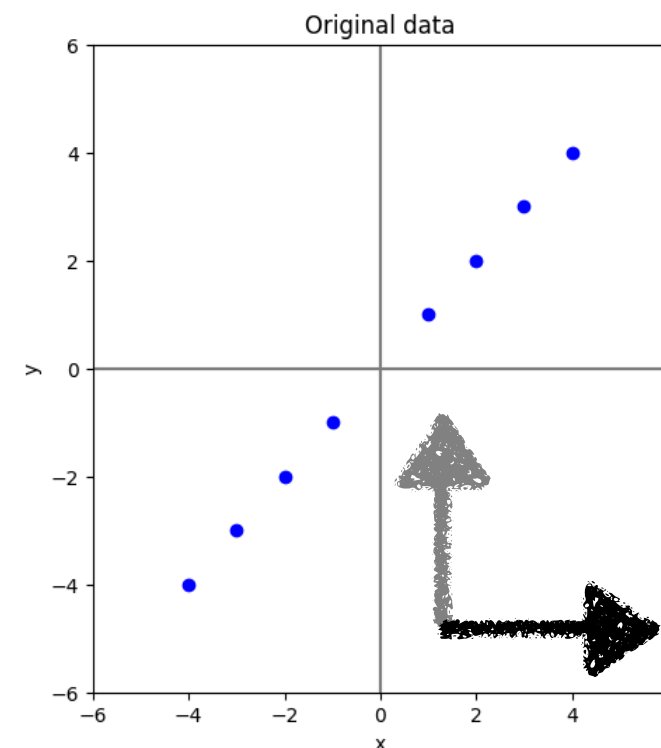
How much you
move along PC_1

How much you
move along PC_2

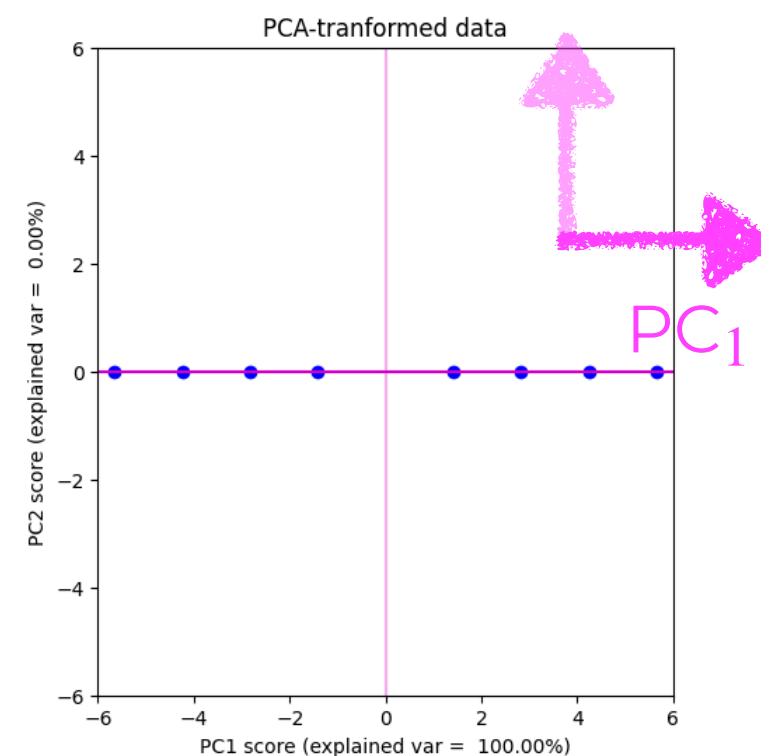
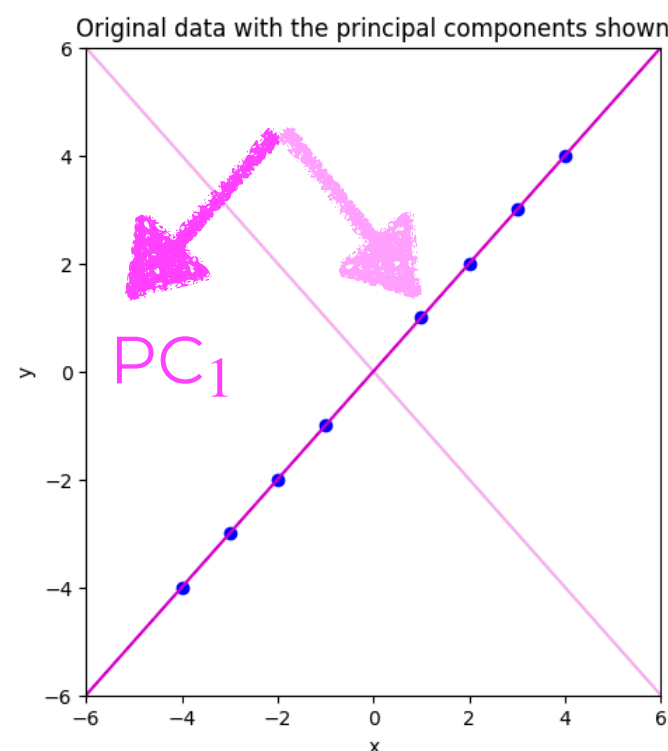
Original space

feature 1 feature 2

sample 1	-4	-4
sample 2	-3	-3
sample 3	-2	-2
sample 4	-1	-1
sample 5	1	1
sample 6	2	2
sample 7	3	3
sample 8	4	4



PCA



PCA

PC₁ score PC₂ score

5.7	0
4.2	0
2.8	0
1.4	0
-1.4	0
-2.8	0
-4.2	0
-5.7	0

Principal Component Analysis (PCA)

`sklearn.decomposition.PCA`

```
class sklearn.decomposition.PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0,
iterated_power='auto', n_oversamples=10, power_iteration_normalizer='auto', random_state=None) \[source\]
```

Attributes:

`components_` : ndarray of shape (n_components, n_features)

Principal axes in feature space, representing the directions of maximum variance in the data. Equivalently, the right singular vectors of the centered input data, parallel to its eigenvectors. The components are sorted by decreasing `explained_variance_`.

$PC_1, PC_2, \dots$

`explained_variance_` : ndarray of shape (n_components,)

The amount of variance explained by each of the selected components. The variance estimation uses `n_samples - 1` degrees of freedom.

$\lambda_1, \lambda_2, \dots$

Equal to `n_components` largest eigenvalues of the covariance matrix of `X`.

New in version 0.18.

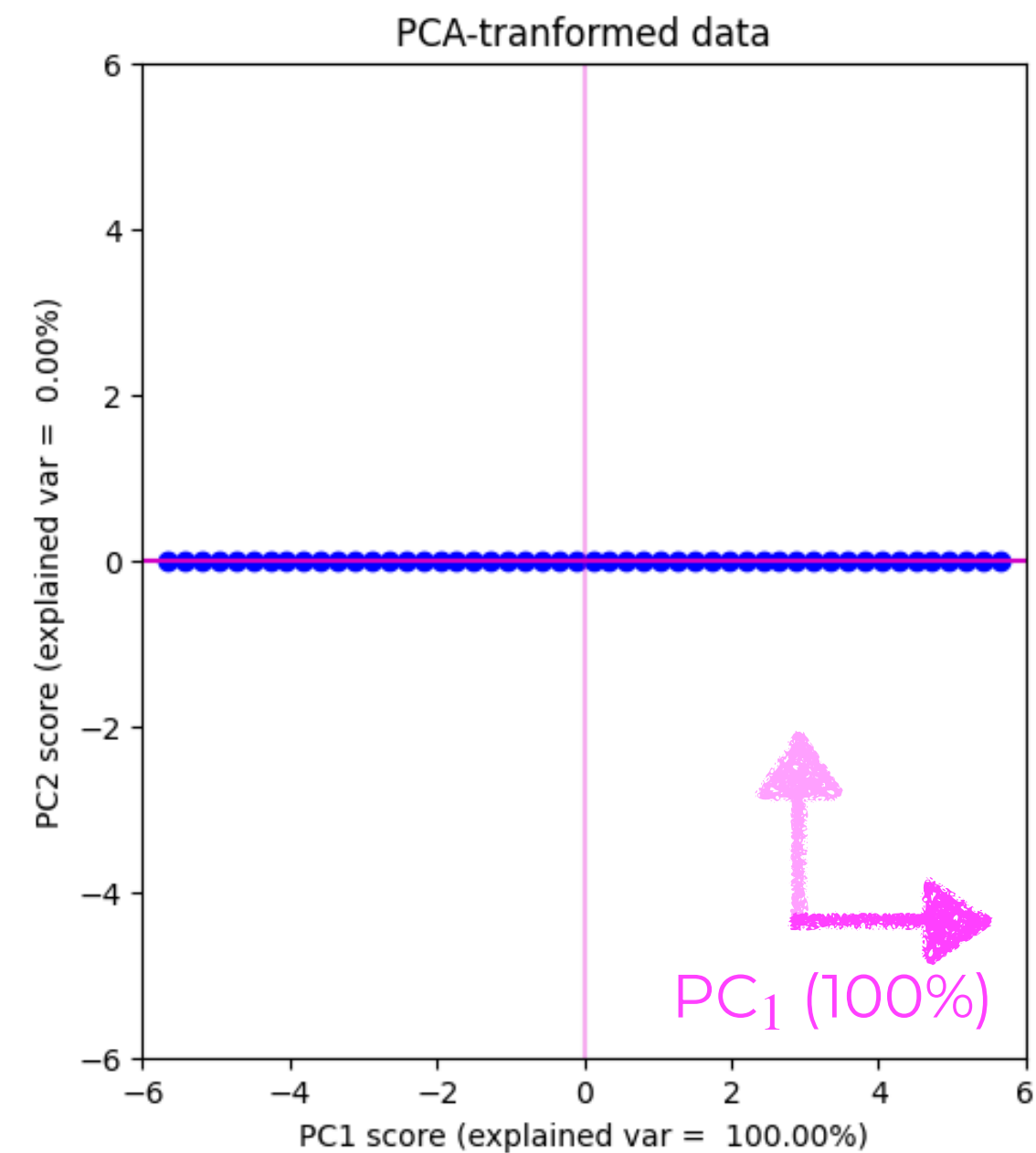
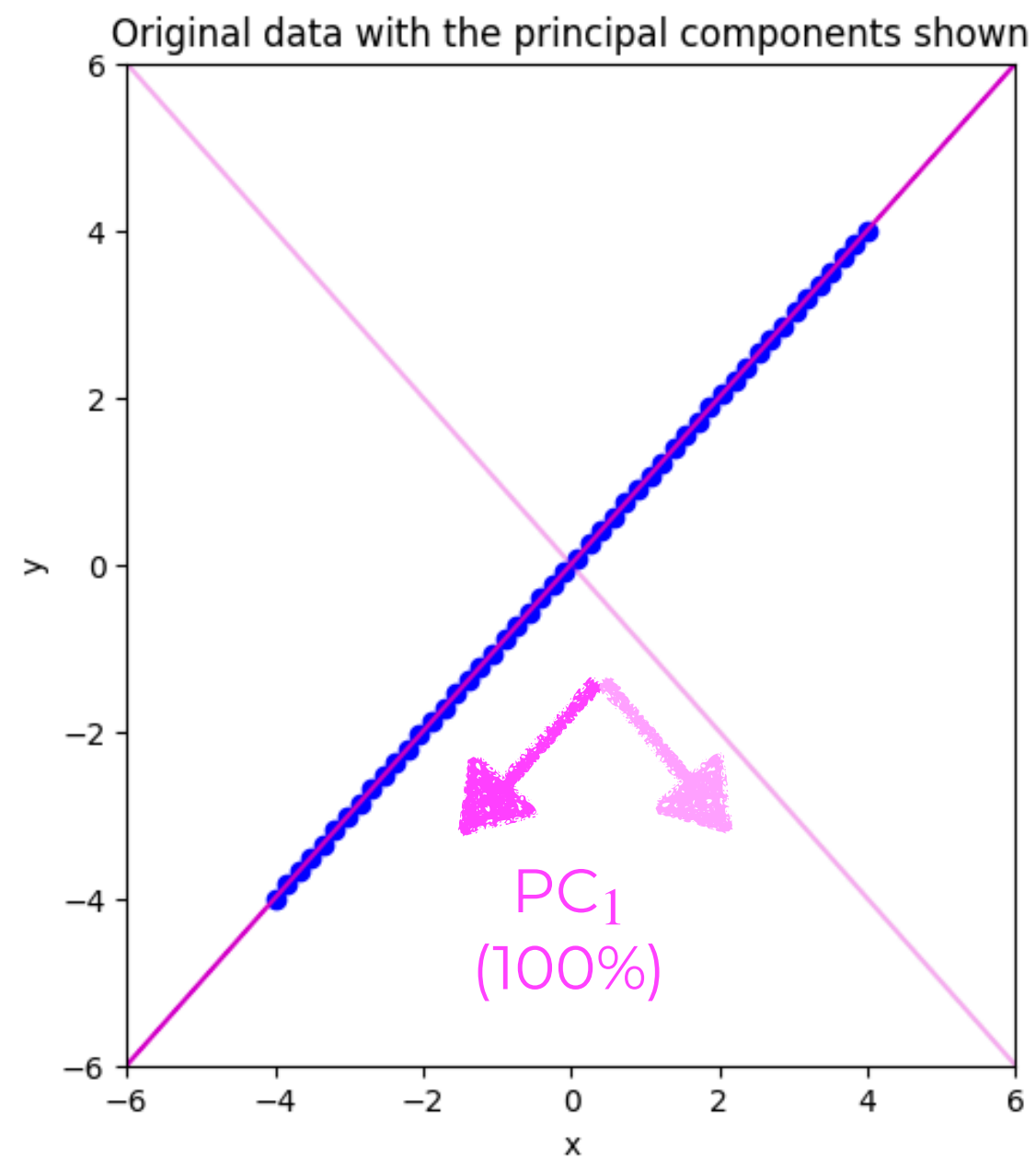
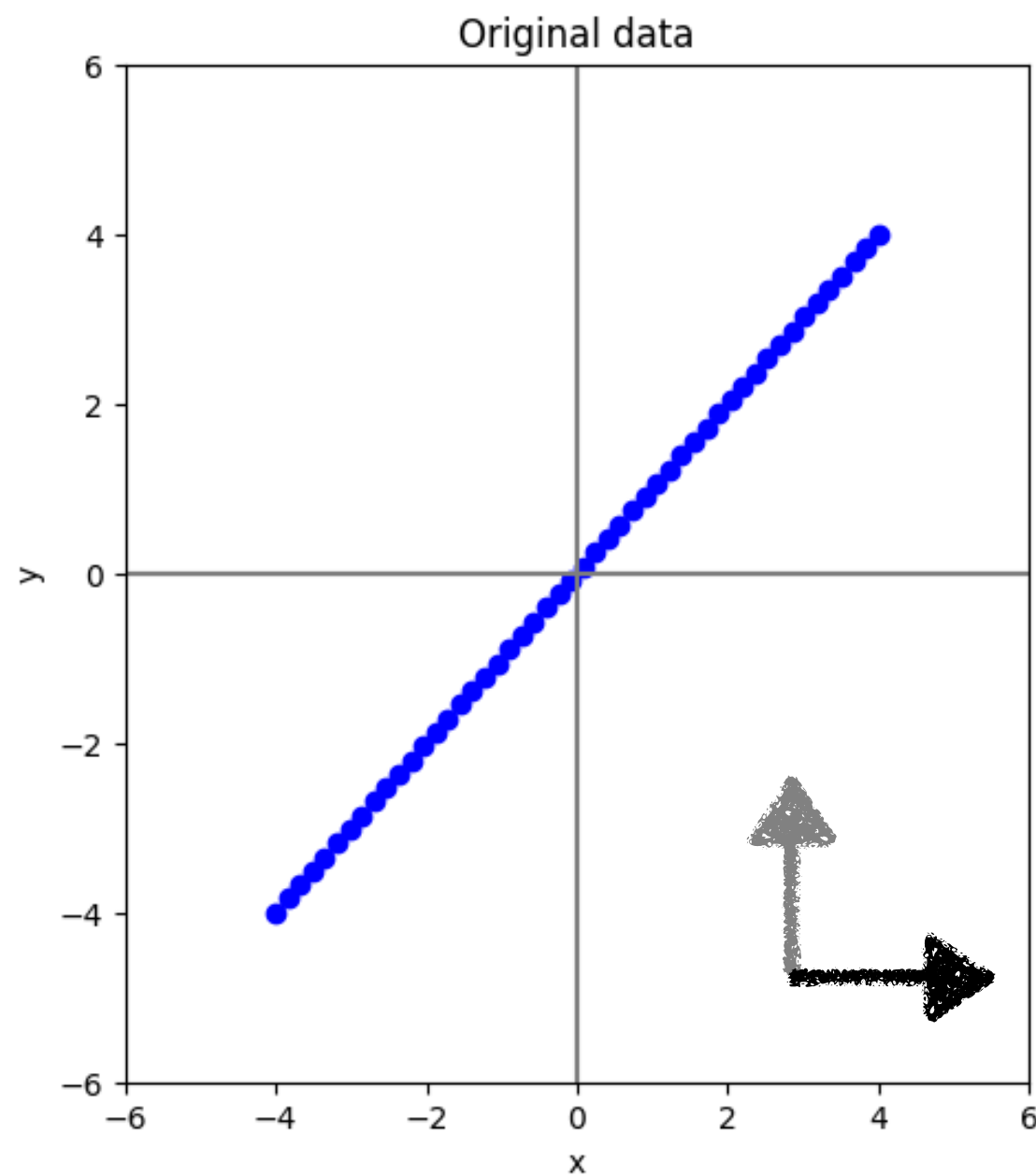
`explained_variance_ratio_` : ndarray of shape (n_components,)

Percentage of variance explained by each of the selected components.

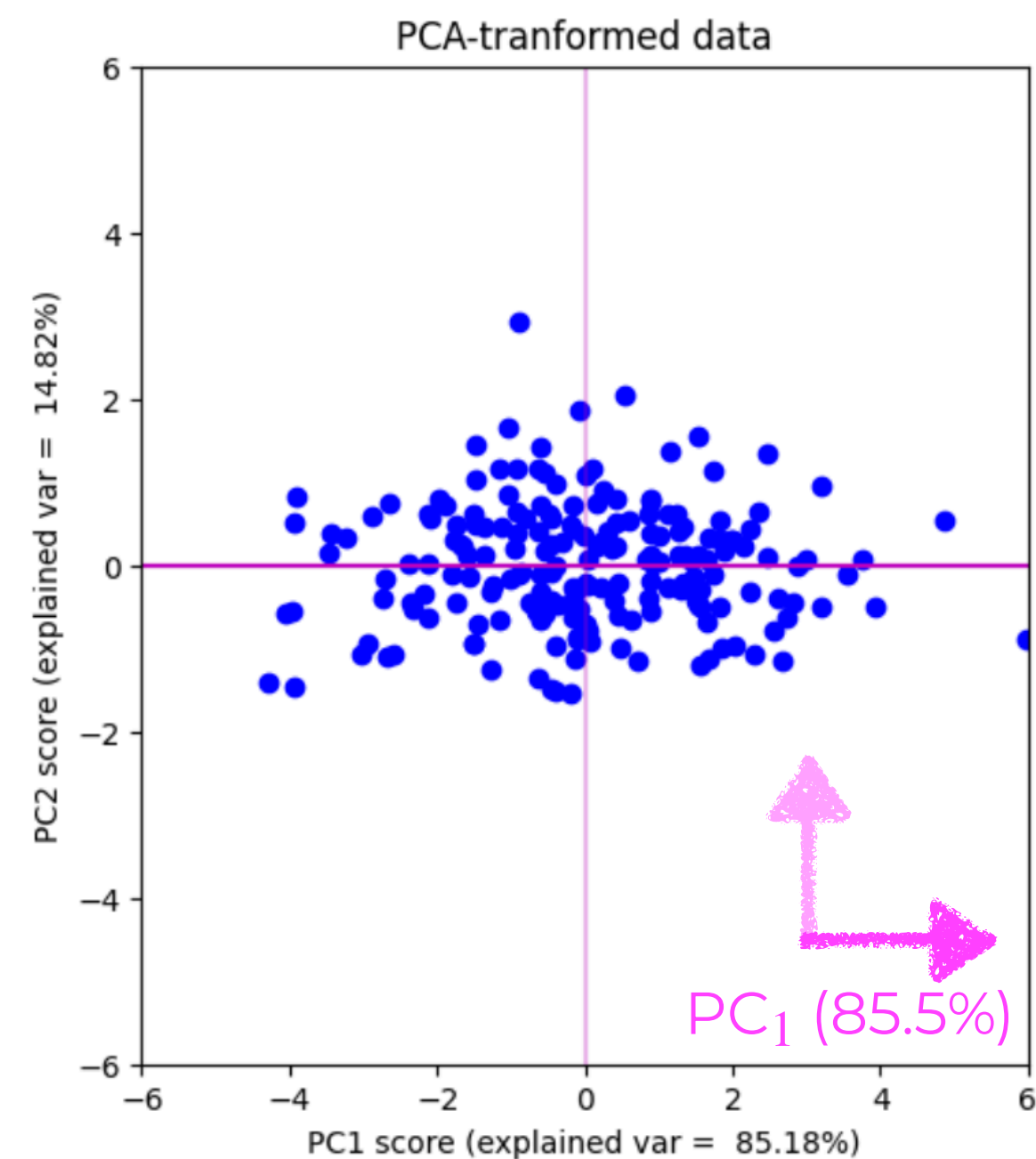
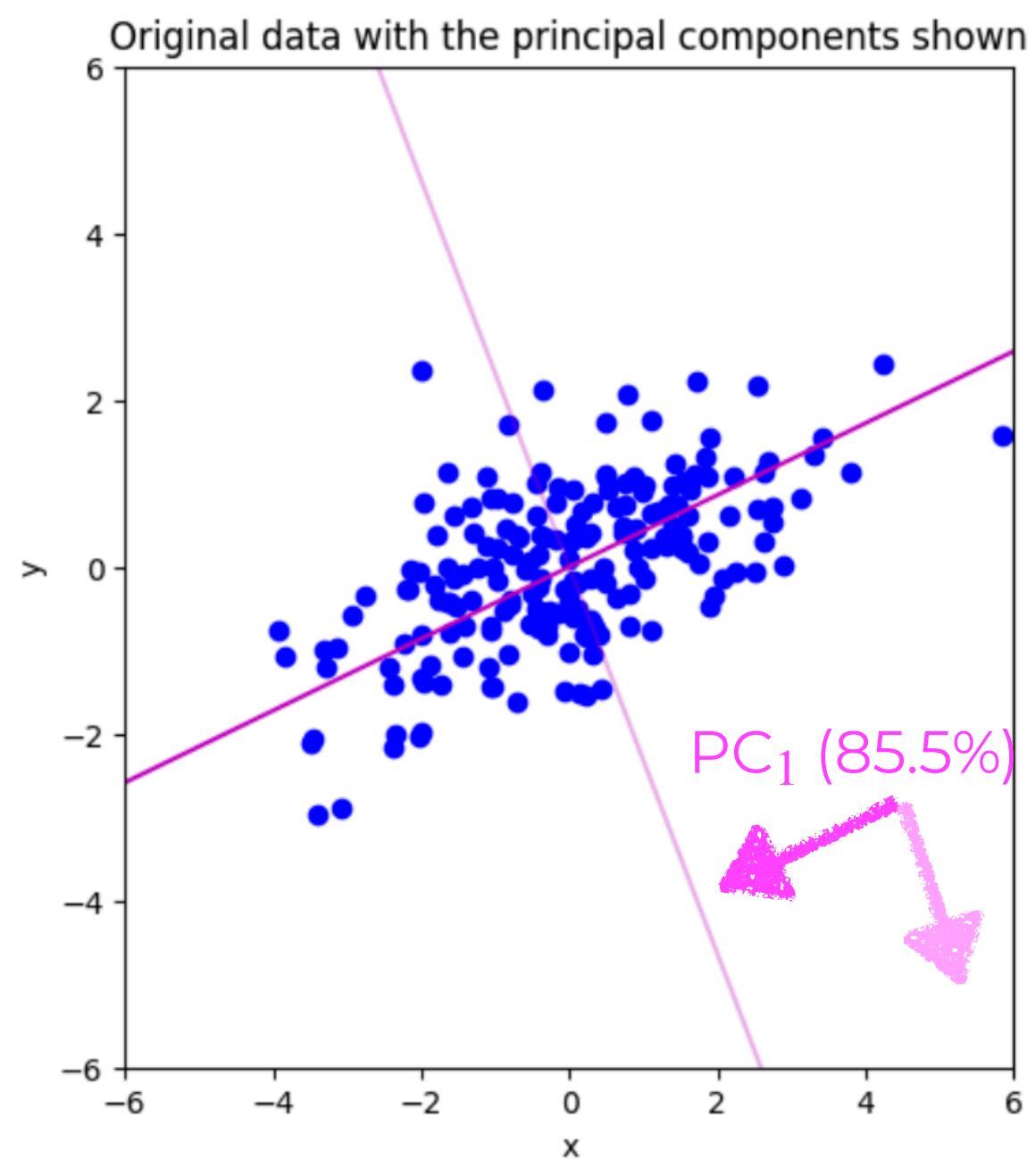
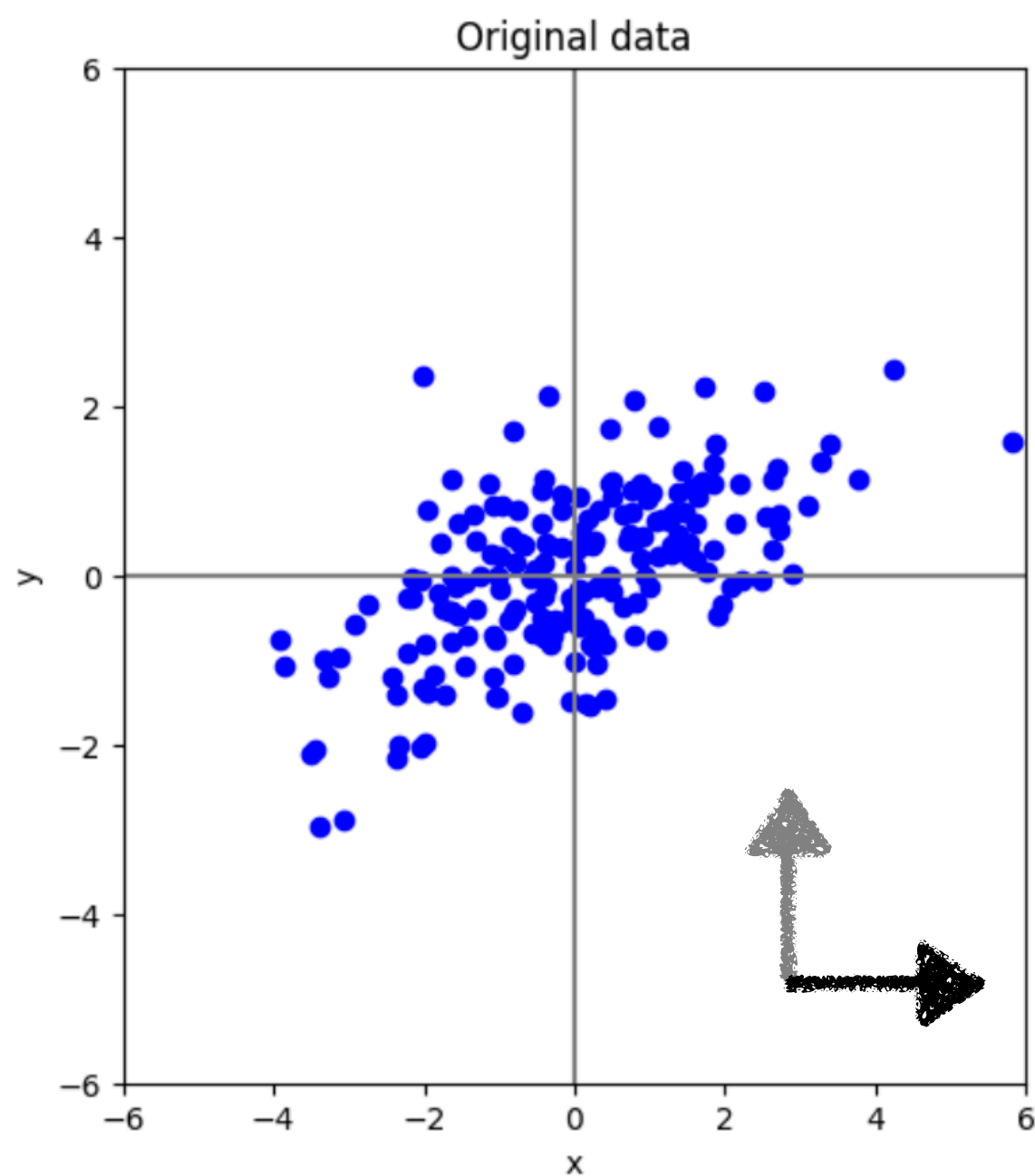
ex. 0.8, 0.2, ...

If `n_components` is not set then all components are stored and the sum of the ratios is equal to 1.0.

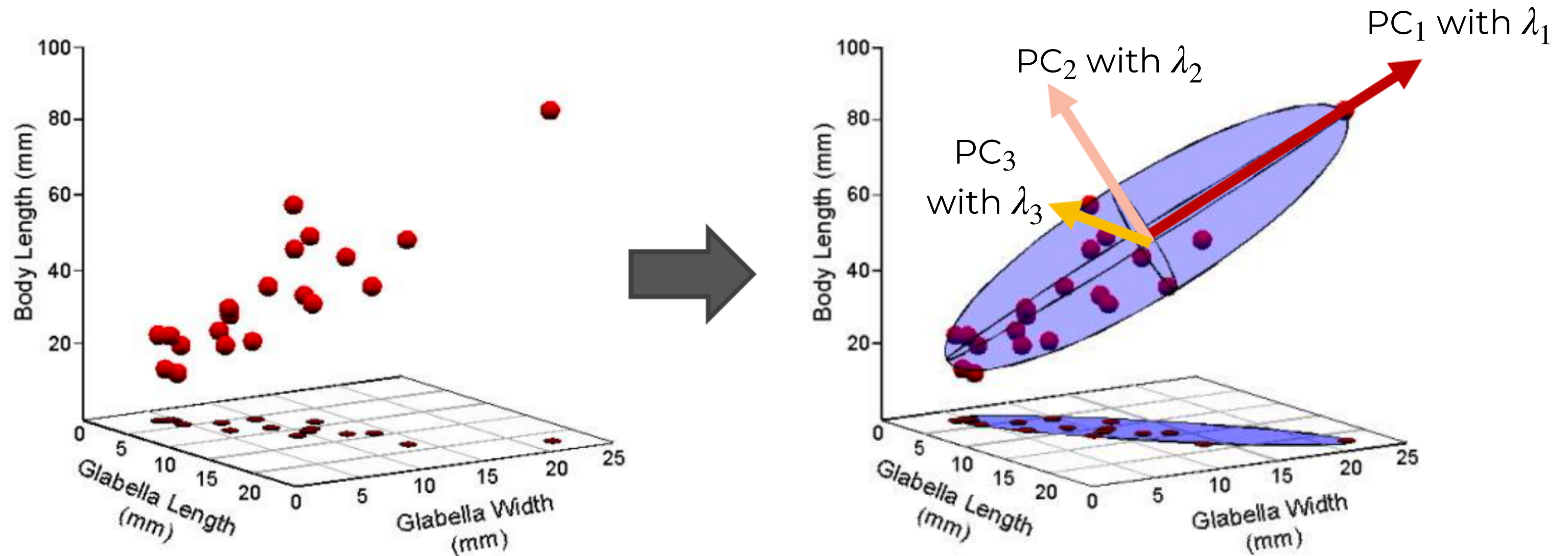
PCA in 2D



PCA in 2D



PCA in 3D



Source: the paleontological association

PCA in High-dimensional Space

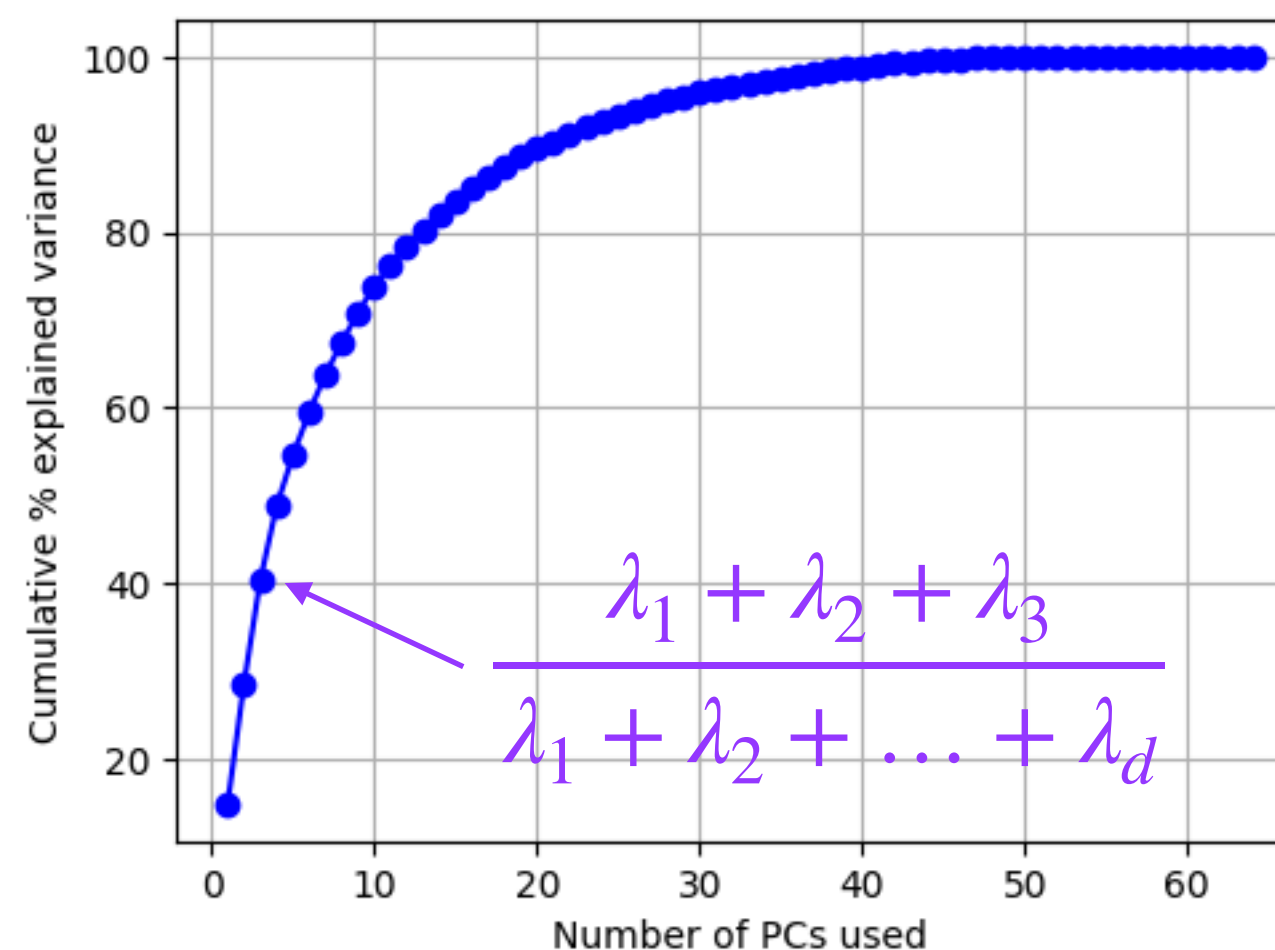
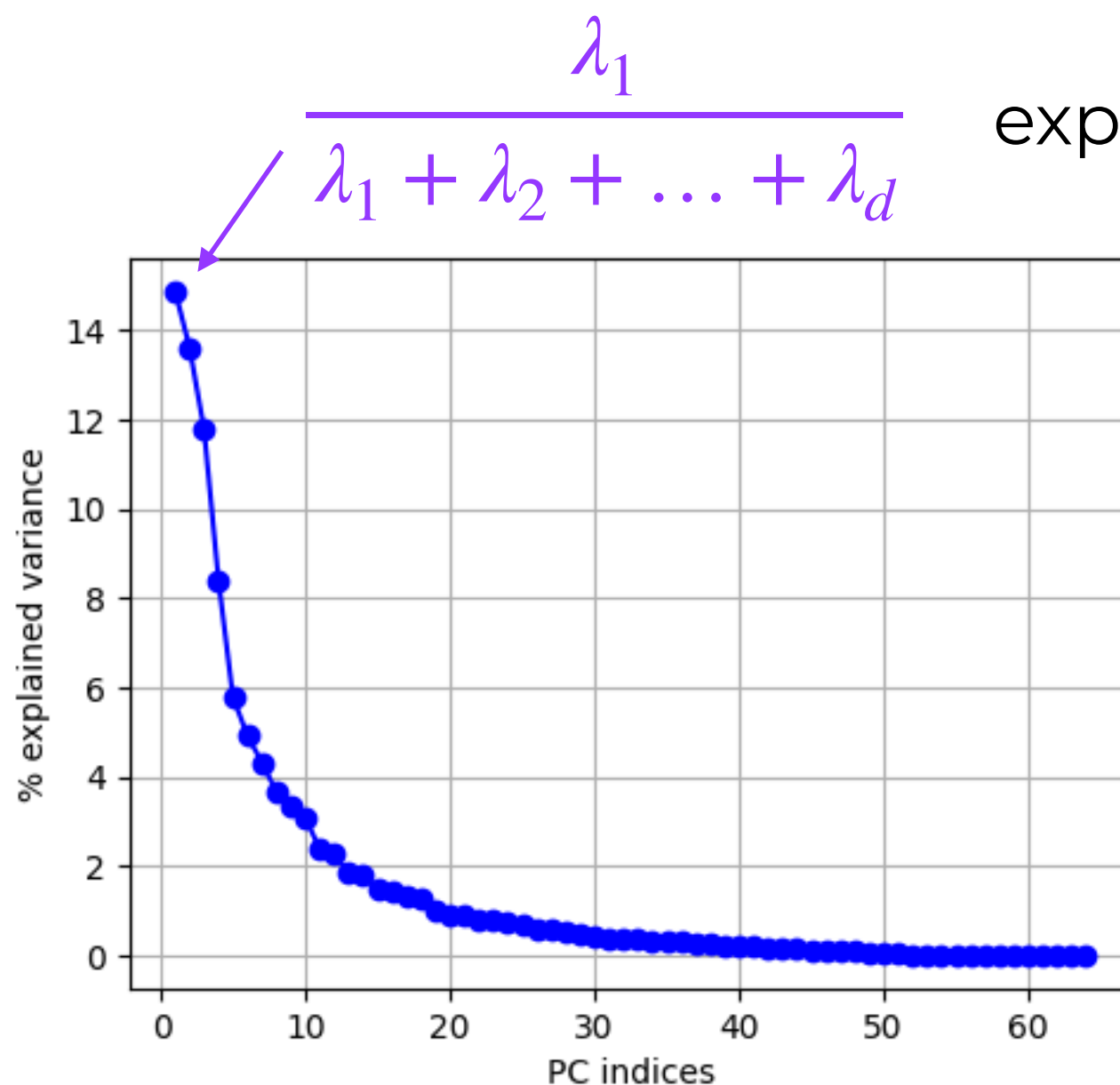
d-dimensional space

d principal components (PCs)

$PC_1, PC_2, \dots, PC_d$

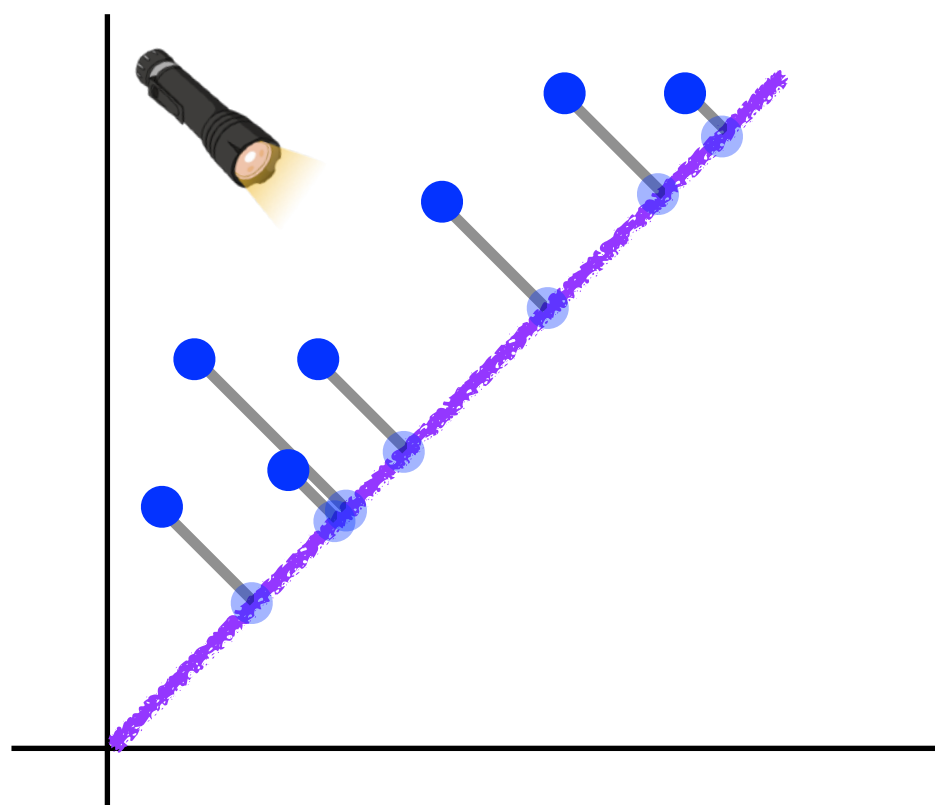
explained variances

$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$



Two Common Definitions of PCA

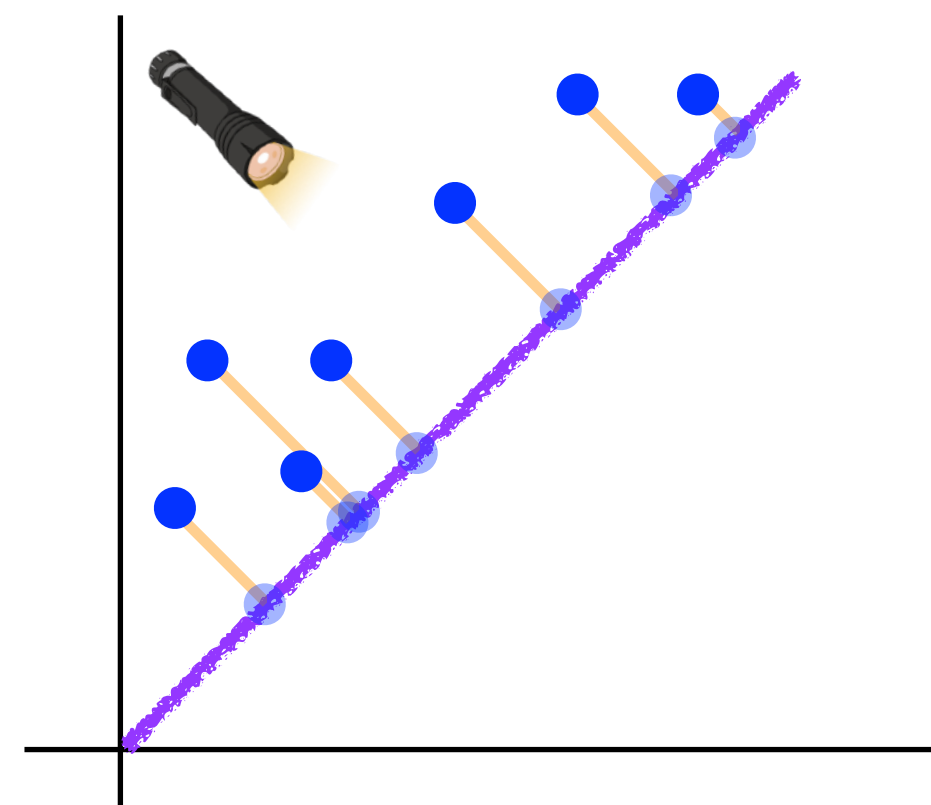
Maximum variance formulation



Maximizes the variance of the projected points



Minimum-error formulation

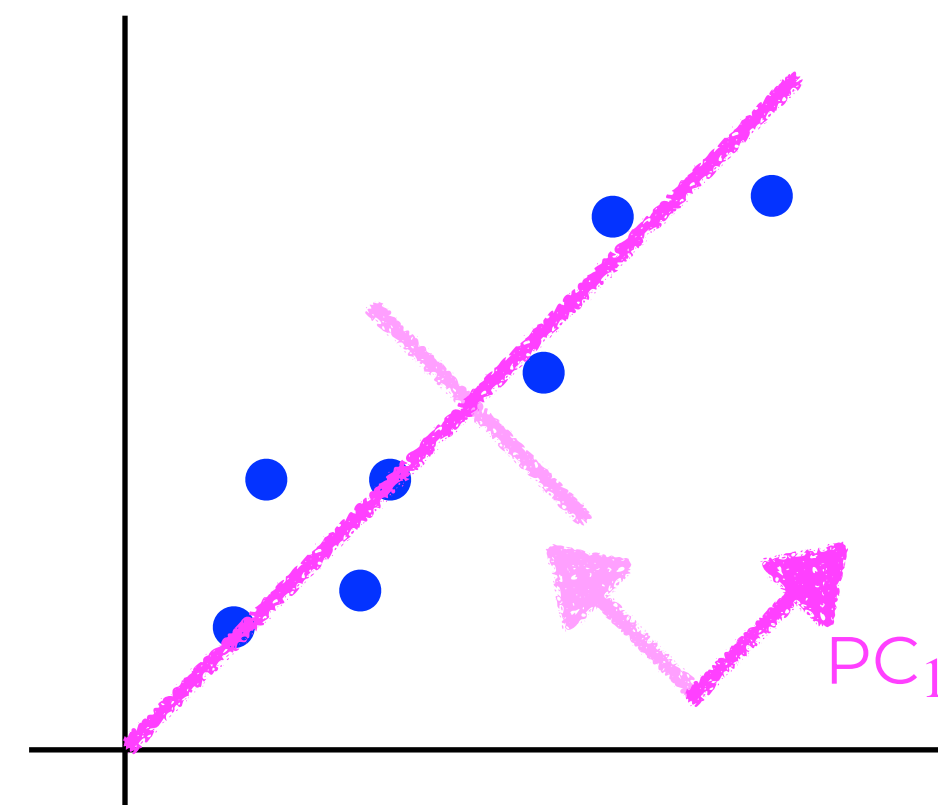
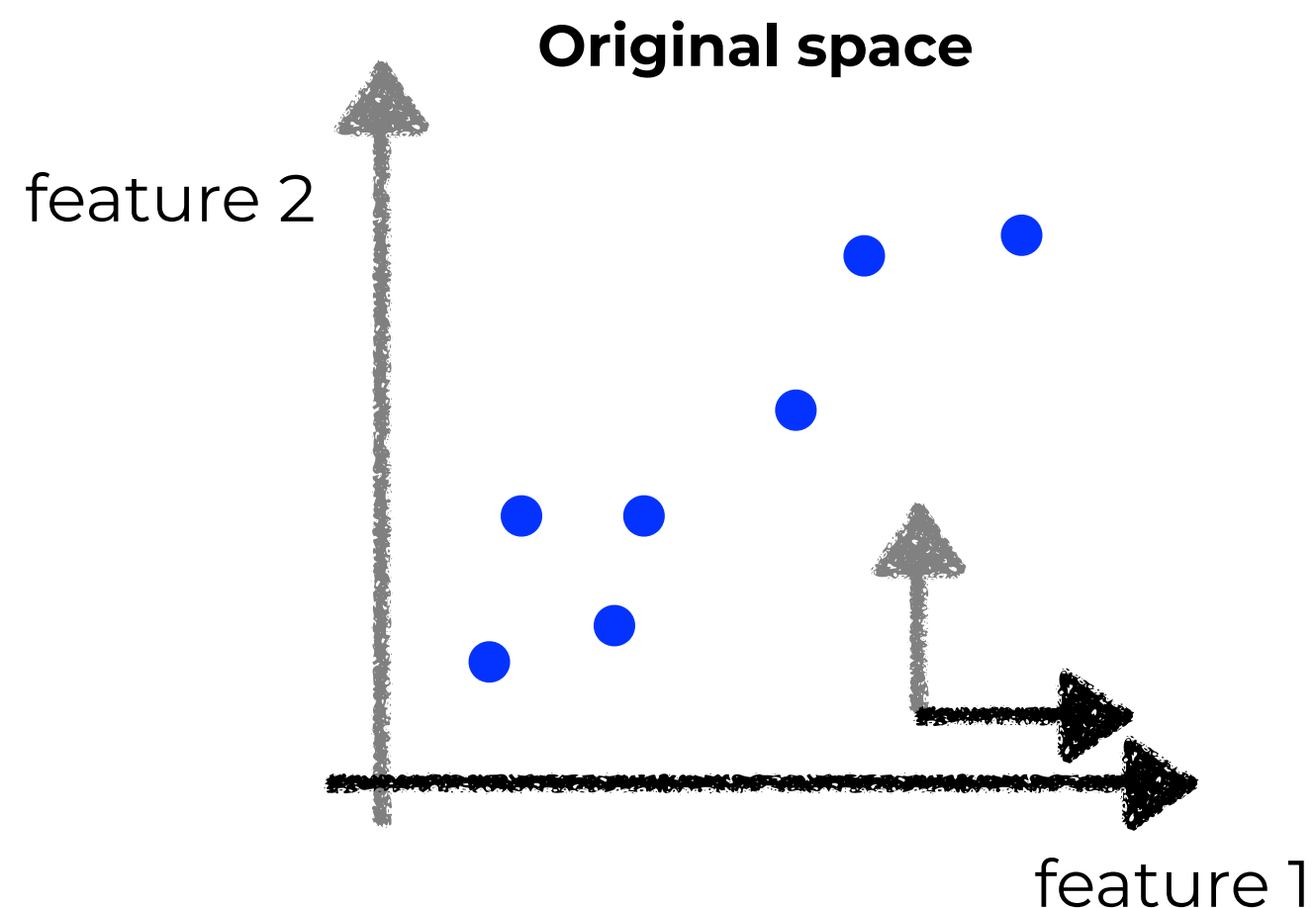


Minimizes the sum-of-squares of the projection errors

$$\text{---}^2 + \text{---}^2 + \text{---}^2 + \dots + \text{---}^2 + \text{---}^2$$

Pros and Cons of PCA

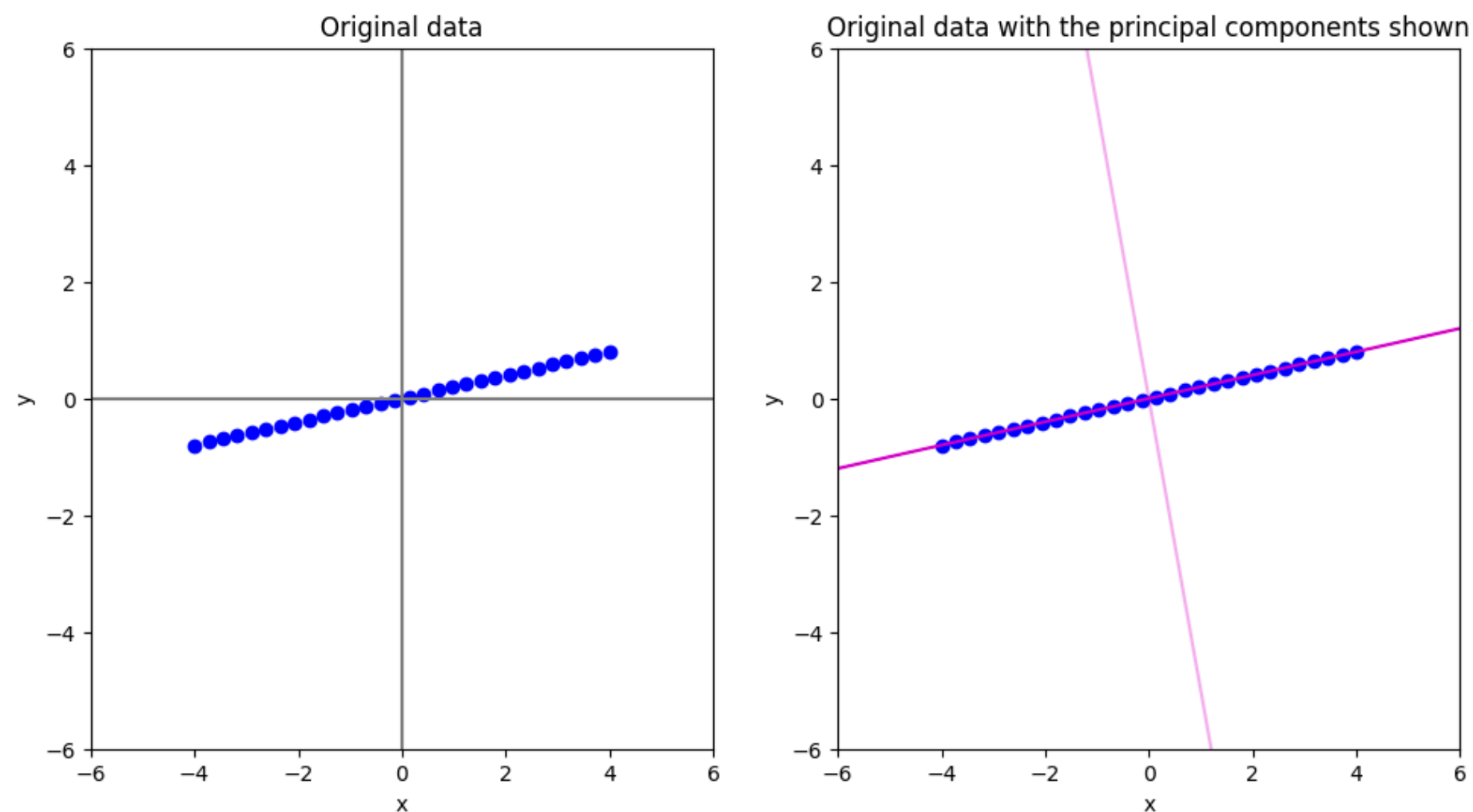
- Each **PC can be interpreted** by looking at how it can be written as a linear combination of the original axes



$$\text{PC}_1 = 0.707 \rightarrow + 0.707$$

Pros and Cons of PCA

- Each **PC can be interpreted** by looking at how it can be written as a linear combination of the original axes



$\rightarrow$
 PC_1

=

0.98

$\rightarrow$

+

0.20

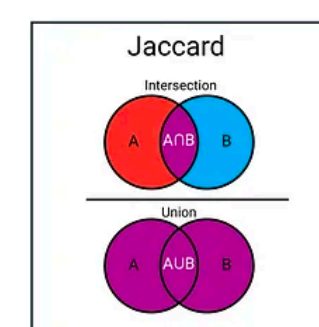
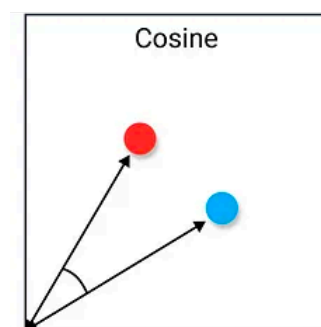
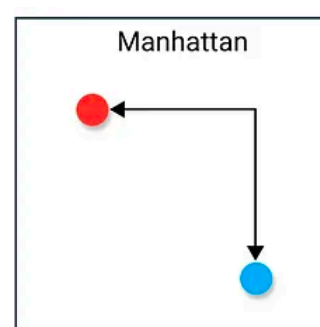
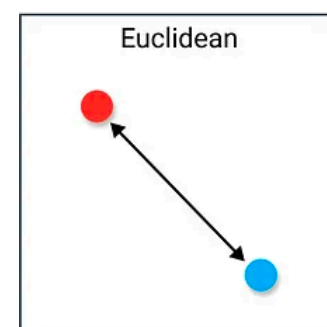
$\uparrow$

Pros and Cons of PCA

- Each **PC can be interpreted** by looking at how it can be written as a linear combination of the original axes

$$\text{PC}_1 = 0.98 \rightarrow + 0.20 \uparrow$$

- Highly correlated features tend to be grouped into the same PC
 - It can be used as a preprocessing step
 - Decorrelate** the variables
 - Perform **dimensionality reduction**
 - We need to be careful with our interpretation
- PCA strictly **preserves the Euclidean distance** between data points, which may not be ideal in many situations



source: <https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa>